Agent-JIT: Dynamic Sub-Schema Projection, Role-Attuned Capability Synthesis, and Linearizable TrueTime Auditing for Enterprise Model Context Protocol

Aniket Agrawal
Google Cloud India Private Limited
Bangalore, India
aniketagrawal@google.com

***Abstract*—The enterprise deployment of autonomous Large Language Model (LLM) agent swarms via the Model Context Protocol (MCP) against mission-critical enterprise systems of record—including Enterprise Resource Planning (ERP) suites such as SAP S/4HANA, CRM platforms such as Salesforce, and distributed database ledgers such as Cloud Spanner—faces an existential architectural bottleneck termed the *Enterprise Schema Explosion and Privilege Leakage Dilemma*. Enterprise backends routinely expose upwards of 40,000 OData entity sets and hundreds of thousands of relational attributes. Statically advertising exhaustive tool catalogs in LLM system prompts induces massive context window bloat (consuming 25,000–50,000 tokens of schema boilerplate for a single enterprise sub-domain, while the full catalog is physically intractable for standard context windows), incurs a 5× prompt latency penalty, provokes severe cross-attention dilution and tool hallucination, and critically bypasses principal authorization by exposing privileged administrative endpoints to unprivileged callers under ambient authority.**

We present Agent-JIT, a sub-millisecond dynamic schema projection, role attenuation, and capability synthesis middleware engine designed for Google Cloud Gemini Enterprise. Agent-JIT coordinates four synchronized pipeline stages: (1) *Principal-Propagated Role Attenuation*, enforcing fail-closed pruning and column-level schema masking based on caller SAP PFCG roles and OAuth/SAML tokens; (2) *Just-In-Time (JIT) Composite Tool Synthesis*, compiling multi-step relational join pipelines (e.g., Accounts Payable 3-way matching across Purchase Orders, Invoices, and General Ledger ACDOCA items) into candidate single-turn execution wrappers $\mathcal{W}_{\text{cand}}$ with Saga and SAP LUW rollback semantics; (3) *Goal-Conditioned Sub-Schema Pruning*, utilizing a heterogeneous capability multigraph combining normalized lexical BM25 and dense semantic embeddings to project budget-bounded relevant tool subsets under a strict token budget with greedy mutual exclusion; and (4) *Cloud Spanner Linearizable TrueTime Auditing*, anchoring partitioned, tamper-evident cryptographic vouchers to physical disk to establish zero-trust execution verification. Empirical evaluation demonstrates: (i) on a persistent disk-backed testbed ($N = 1{,}000$ trials across 10 to 500 ERP tools), an 84.80% macro-averaged token reduction (peak 98.20%), 100.00% Tool Retrieval Recall@5, an in-process middleware compute and local SQLite WAL commit protocol latency of $p_{50} = 1.209$ ms (`tools/list`) and 1.918 ms roundtrip under warm query embeddings (29.6 ms including live non-blocking Vertex AI `text-embedding-004` RPC), an 82.82% overall wire payload reduction, and 100.00% rejection of 150 automated socket injection probes; and (ii) in live Google Cloud Gemini Enterprise foundation model benchmarks (Gemini 2.5 Flash and Gemini 2.5 Pro), an 89.09% macro-averaged prompt token reduction (peaking at 97.41% at catalog size $M = 100$ tools: 4,792 tokens down to 124 tokens), 100.00% single-turn composite resolution, and 0.00% unauthorized tool invocation across both an interactive red-team sanity check ($0/8$ vs. $3/8$ trials, one-sided Fisher's $p = 0.1000$) and an expanded $N = 50$ standardized adversarial prompt-injection suite ($0/50$ [0.00%, 95% CI $[0.00\%, 7.11\%]$] vs. $18/50$ [36.00%] in unattenuated monolithic baselines, exact Fisher's $p = 4.6 \times 10^{-7}$).

***Index Terms*—Model Context Protocol, FastMCP, Gemini Enterprise, Cloud Spanner, TrueTime, SAP S/4HANA, Dynamic Schema Projection, Privilege Attenuation, Zero-Trust Architecture.**

I. Introduction

Autonomous AI agent systems orchestrated by production foundation models—such as Google Cloud Vertex AI Gemini 2.5 Flash and Gemini 2.5 Pro—are rapidly transitioning from conversational chat interfaces to transactional execution engines embedded within enterprise systems of record [1]–[3]. To standardize how foundation models discover, negotiate, and invoke external software tools, the open-standard Model Context Protocol (MCP) [4] and high-performance server implementations such as FastMCP have gained pervasive industry adoption.

However, transitioning MCP from developer toolsets to enterprise systems of record—including Enterprise Resource Planning (ERP) suites exposing OData v4 CSDL schemas such as SAP S/4HANA [5], CRM platforms, and distributed database ledgers such as Cloud Spanner [6]—exposes fundamental architectural limitations:

1) **The Schema Explosion Problem:** Enterprise systems do not expose 10 or 20 discrete tools; a standard SAP S/4HANA deployment exposes over 40,000 OData entity sets, each with dozens of typed attributes, associations, and navigation paths [5]. Serializing even 50 to 100 enterprise tool schemas in verbose OpenAPI JSON format consumes 25,000 to 50,000 tokens per prompt turn (or $\approx 4{,}800$ tokens even under aggressive schema minification), exhausting context space, inflating per-turn

token costs, and causing 3× to 5× prompt processing latency spikes, while serializing the full catalog requires millions of tokens and is physically impossible.

2) **Attention Dilution, Hallucination, and Relational Blindness:** As candidate tools advertised in the context window scale beyond 50, LLM tool-calling and parameter-binding accuracy degrades precipitously [7]–[9]. Cross-attention mechanisms become diluted across thousands of irrelevant parameter definitions (the "Lost in the Middle" phenomenon [7]), leading to erroneous parameter binding, selection of invalid endpoints, or outright hallucination of non-existent actions. While dense neural retrievers (ToolLLM [8]), instruction-tuned decoders (Gorilla [9]), hierarchical clustering (AnyTool [10]), co-occurrence graph retrievers (ToolNet [11] and COLT [12]), active MCP discovery agents (ScaleMCP [13] and MCP-Zero [14]), passive pre-invocation MCP vector retrievers (RAG-MCP [15]), and protocol-native MCP Progressive Tool Discovery [4] attempt to filter tool catalogs, existing approaches either require secondary LLM discovery turns or score flat tool descriptions in isolation without enforcing enterprise RBAC column masking or token-budgeted set-packing invariants. Crucially, isolated docstring retrieval exhibits relational foreign-key blindness on enterprise benchmarks [16]–[18]: standard retrievers fail to surface essential upstream dependencies (e.g., omitting `A_PurchaseOrder` when a query mentions only `A_SupplierInvoice`).

3) **Privilege Leakage and the Confused Deputy Problem:** In standard MCP architectures, the tool catalog advertised to the model is static and server-centric. An unprivileged clerk querying procurement data is exposed to schemas for administrative treasury disbursement runs (e.g., SAP F110 wire transfers) or confidential payroll tables. In enterprise deployments, this vulnerability manifests across two architectural settings: (i) in architectures where tool gateways utilize shared service accounts possessing ambient authority, backend Role-Based Access Control (RBAC) [19] inspects only the elevated service identity, falling victim to the classical Confused Deputy Problem [20] when adversarial prompt injections [21], [22] coerce the model into emitting privileged calls; and (ii) even in architectures with downstream identity propagation (e.g., IETF RFC 7522 SAML 2.0 Bearer Assertions [23], RFC 8693 OAuth 2.0 Token Exchange [24], or MCP OAuth 2.1 audience-bound resource indicators under IETF RFC 9728 and RFC 8707 [4]), coarse server-level token scopes still advertise all schemas within an authorized MCP server, causing prompt-layer capability leakage and exposing restricted attributes. While contemporary Information-Flow Control (IFC) and privilege control architectures (such as CaMeL [25], FIDES [26], Progent [27], SEAgent [28], and MCPSec [29]) investigate application-layer taint tracking or planning-stage checks, Agent-JIT enforces structural capability containment, column-level schema masking, and fail-closed catalog scoping directly at the transport layer prior to prompt serialization; fine-grained intra-role parameter manipulation is orthogonal and addressed by runtime parameter inspection or data-provenance tracking.

4) **Multi-Hop Conversation Churn:** Enterprise workflows are inherently relational. In analytical reconciliation inquiries—such as an Accounts Payable (AP) "3-way match" traversing Purchase Orders (`API_PURCHASEORDER_PROCESS_SRV`), Goods Receipt line items (`API_MATERIAL_DOCUMENT_SRV`), Supplier Invoices (`API_SUPPLIERINVOICE_PROCESS_SRV`), and Universal Journal postings (`ACDOCA`) [5]—standard MCP and iterative multi-turn planners (e.g., RestGPT [30] and ReAct [31]) force the LLM into 3 to 5 sequential back-and-forth conversational round trips, compounding latency to over 5.5 seconds and multiplying the risk of premature agent aborts. While compiler-inspired agent planners (LLMCompiler [32]), transactional runtimes (SagaLLM [33] and GoEX [34]), and protocol-native Programmatic Tool Calling [4] allow multi-tool orchestration or sandboxed code generation, they either introduce secondary model invocations and multi-second scheduling overhead or require the LLM to synthesize imperative join and rollback code from scratch without OData CSDL foreign-key pushdown or pre-verified Saga/LUW compensation invariants. Conversely, in multi-step mutating workflows, executing sequential atomic tools without transactional boundaries risks leaving systems in partially committed, corrupt states upon mid-execution failure.

5) **Absence of Linearizable Execution Auditing in Multi-Tenant Agent Swarms:** In enterprise environments, autonomous agents make non-deterministic routing decisions across distributed microservices. Conventional application logging relies on asynchronous, uncoordinated local logs vulnerable to log injection, non-deterministic race conditions between sub-schema projection and tool dispatch, and clock drift. Ephemeral server logs cannot provide tamper-evident cryptographic proof that an executed action was strictly authorized and projected at the moment of reasoning, failing regulatory compliance (e.g., SOC 2, ISO 27001) and leaving enterprise ledgers without external linearizability.

To resolve this multi-faceted challenge, this paper introduces **Agent-JIT**, a hardware-speed dynamic schema projection, security attenuation, and composite synthesis middleware engine natively integrated into Google Cloud Gemini Enterprise and Cloud Run FastMCP servers.

The core contributions of this work are as follows:

- **Principal-Propagated Role Attenuation:** A fail-closed security gateway that maps caller enterprise credentials (SAML 2.0 / OAuth bearer assertions and SAP PFCG roles) directly to entity sets and field masks, pruning unau-

thorized tools prior to LLM visibility and mathematically mitigating confused deputy attacks.

- **Just-In-Time Composite Tool Synthesizer:** An in-memory AST compiler that inspects multi-step relational workflows: performing dynamic foreign-key join push-down for read-only analytical reconciliation queries to eliminate multi-turn conversational latency, and compiling parameterized declarative workflow templates with Saga-style compensating transactions [35], [36] and SAP Logical Unit of Work (LUW) rollback boundaries for mutating operations, collapsing multi-turn conversational latency into a unified execution pipeline.
- **Goal-Conditioned Heterogeneous Capability Multi-graph:** A capability multigraph $G = (V_T \cup V_E, E_R \cup E_N)$ that combines normalized BM25 lexical matching and dense embeddings, propagates semantic relevance across relational navigation paths, and solves a bounded-knapsack projection with mutual exclusion under strict context budgets ($\leq$ 2,000 tokens) with 100.00% Tool Retrieval Recall@5.
- **Cloud Spanner Linearizable TrueTime Auditing:** A partitioned, tamper-evident cryptographic ledger recording two-stage state vouchers ($\mathcal{V}_{\text{proj}}$ and $\mathcal{V}_{\text{inv}}$) linked via sequential hash chains backed by TrueTime linearizable commit timestamps [6] and distributed event ordering principles [37], guaranteeing zero-trust containment [38] without cross-tenant write contention.
- **Empirical Multi-Tier Verification:** A comprehensive empirical evaluation combining: (i) an offline physical systems testbed ($N = 1{,}000$ trials) over official MCP JSON-RPC 2.0 streaming on persistent disk SQLite WAL databases emulating SAP S/4HANA schemas and Spanner commit logs (plus 150 public tasks adapted from CRMArena-Pro [16], MCP-Universe [17], and MCP-Bench [18]), confirming an 84.80% macro-averaged offline schema token reduction (peaking at 98.20% at catalog size $M = 500$), an 82.82% overall wire payload reduction, in-process middleware compute and local SQLite WAL commit protocol latency of $p_{50} = 1.209$ ms for `tools/list` (29.6 ms under online embedding RPC) and 1.918 ms total MCP roundtrip, live Google Cloud Memorystore for Redis ring-buffer verification ($p_{50} = 0.48$ ms) and multi-region Cloud Spanner Paxos 2PC commit latency ($p_{50} = 6.42$ ms), and 100% socket injection rejection ($N = 150, p < 10^{-15}$); and (ii) a live Google Cloud Vertex AI / Gemini Enterprise benchmark across Gemini 2.5 Flash and Gemini 2.5 Pro, demonstrating an 89.09% macro-averaged prompt token reduction (peaking at 97.41% at $M = 100$), 100% single-turn AP 3-way match resolution, and 0.00% unauthorized tool dispatch across both interactive sanity-check trials (0/8 vs. 3/8, $p = 0.1000$) and an expanded $N = 50$ standardized prompt-injection benchmark (0/50 vs. 18/50, $p = 4.6 \times 10^{-7}$) [22], [39], [40].

Host Protocol Integration & Operational Scope: Standard MCP clients negotiate tools once at connection initialization and may issue parameterless `tools/list` requests ($q = \emptyset$). Agent-JIT supports two well-defined operational modes across client types: (1) *Unmodified Off-the-Shelf MCP Clients:* At connection initialization, Stage 1 role attenuation first prunes the server catalog M to the principal's authorized subset $\mathcal{T}_{\text{auth}}$. Whenever $\Phi(\mathcal{T}_{\text{auth}}) \leq B$ (where $\Phi(\mathcal{T}_{\text{auth}}) \triangleq \sum_{t \in \mathcal{T}_{\text{auth}}} \text{Cost}(t)$ and $B = 2{,}000$ tokens)—which holds for specialized enterprise roles where Stage 1 alone prunes $> 70\%$ of the catalog—the entire $\mathcal{T}_{\text{auth}}$ fits within B, achieving 100% recall on unmodified clients without requiring q. When a broad role yields $\Phi(\mathcal{T}_{\text{auth}}) > B$ and $q = \emptyset$, Stage 3 ranks $t \in \mathcal{T}_{\text{auth}}$ by prior value density $P_{\text{prior}}(t \mid \mathcal{R}_{\text{pfcg}})/\text{Cost}(t)$—where $P_{\text{prior}}(t \mid \mathcal{R}_{\text{pfcg}}) \triangleq \max_{r \in \mathcal{R}_{\text{pfcg}}:(r,\text{srv}_t,\text{ent}_t,op_t) \in \mathcal{A}_{\text{auth}}} \frac{N_{\text{inv}}(t,r)+1}{\sum_{t' \in \mathcal{T}_r} N_{\text{inv}}(t',r)+|\mathcal{T}_r|}$ aggregates Laplace-smoothed empirical invocation frequencies $N_{\text{inv}}(t, r)$ from the tenant's Cloud Spanner audit ledger over the role-authorized tool subset $\mathcal{T}_r$ once $\sum_{t' \in \mathcal{T}_r} N_{\text{inv}}(t', r) \geq N_{\text{warm}} = 50$ historical invocations, with cold-start fallback for < 50 observations to normalized undirected entity degree $\deg_{E_N}(\text{ent}_t)/\text{Cost}(t)$ over E_N restricted to $\mathcal{T}_{\text{auth}}$ and deterministic tie-breaking by minimal $\text{Cost}(t)$, then lexicographical id_t—and terminates server-side cursor pagination (`nextCursor = null`) once cumulative emitted tokens reach B. For unmodified clients when $\Phi(\mathcal{T}_{\text{auth}}) > B$, Agent-JIT optionally exposes a compact discovery meta-tool (`mcp_jit_discover_tools`, pre-committed inside $\mathcal{V}^{(1)}_{\text{proj}}.\mathcal{D}_{\text{proj}}$ while reserving $B - \text{Cost}(\texttt{mcp_jit_discover_tools})$ for initial domain tools) that accepts a natural-language task query, runs Stages 2–3 to refresh $\mathcal{T}_{\text{proj}}$, and emits `notifications/tools/list_changed` (trading one extra LLM discovery turn on broad-role cold starts; all recall and single-turn synthesis benchmarks in Section IV evaluate the goal-conditioned mode below). In legacy client compatibility mode where a connection-level voucher pointer $\mathcal{V}^{\text{conn}}_{\text{proj}}$ (initialized to $\mathcal{V}^{(1)}_{\text{proj}}$ and updated to $\mathcal{V}^{(i)}_{\text{proj}}$ whenever `notifications/tools/list_changed` triggers a refreshed `tools/list`) is reused across K subsequent `tools/call` requests that omit `_meta.voucher_id`, the dedicated 1:1 session transport resolves v_{id} to $\mathcal{V}^{\text{conn}}_{\text{proj}}$ and chains each invocation $k \in \{1, \ldots, K\}$ sequentially via $\mathcal{C}^{(\tau,s)}_{i,\text{head}} \leftarrow \mathcal{C}^{(\tau,s)}_{i,\text{inv},k} = \text{SHA-256}(\mathcal{C}^{(\tau,s)}_{i,\text{head}} \parallel \text{ToolID}^{(i,k)}_{\text{inv}} \parallel t^{(i,k)}_{\text{exec}} \parallel \Sigma^{(i,k)}_{\text{inv}})$ while still enforcing tool-set membership $\text{ToolID}_{\text{inv}} \in \text{IDs}(\mathcal{V}^{\text{conn}}_{\text{proj}}.\mathcal{D}_{\text{proj}})$ and column masks $\mathcal{M}_{\text{restr}}$; in strict zero-trust mode (`require_explicit_voucher = true`), omitting `_meta.voucher_id` is rejected immediately with -32001 onto $\mathcal{C}^{(\tau,s)}_{i,\text{probe}}$. (2) *Goal-Aware Enterprise Host Orchestrators:* In Google Cloud Gemini Enterprise host proxies and custom orchestrators, the host proactively issues `tools/list` at the inception of each reasoning turn with per-turn goal metadata (`params._meta.user_goal` or `X-Agent-Goal`) and propagates explicit v_{id} tokens on `tools/call`, while `notifications/tools/list_changed` handles out-of-band SCIM role revocations (RFC 7644 [41]) and

dual-custody sign-off completions [4].

II. System Architecture

The Agent-JIT architecture operates as an inline, low-latency middleware interceptor positioned between the LLM Reasoning Loop in Google Cloud Gemini Enterprise and backend enterprise tool providers, as illustrated in Fig. 1.

A. Principal-Propagated Role Attenuation (Stage 1)

When an enterprise request is initiated, the caller's SAML 2.0 assertion or OAuth JWT bearer token is forwarded to the Agent-JIT engine. The principal security profile contains identity attributes, tenant boundary identifiers, and assigned SAP PFCG security roles (e.g., `Z_FI_AUDITOR_READ`, `Z_MM_PURCHASING_CLERK`):

Definition 1 (Principal Authorization Matrix)**.** *A principal $\mathcal{P}$ is defined as a tuple:*

$$\mathcal{P} = \langle u, \tau, \mathcal{R}_{pfcg}, \mathcal{S}_{auth}, \mathcal{E}_{auth}, \mathcal{O}_{auth}, \mathcal{M}_{restr}, \mathcal{A}_{auth} \rangle \quad (1)$$

where u is the enterprise user ID, τ is the tenant identifier, $\mathcal{R}_{pfcg}$ is the set of assigned PFCG roles, $\mathcal{S}_{auth}$ and $\mathcal{E}_{auth}$ are authorized OData services and entity sets, $\mathcal{O}_{auth} \subseteq \{READ, QUERY, CREATE, UPDATE, DELETE, EXECUTE\}$ is the permitted operation set, $\mathcal{M}_{restr}$ is the field-level restriction mask (indexed by entity-qualified attribute keys `<Entity>.<Attr>` and global wildcards `*.<Attr>` to prevent cross-entity namespace collisions), and $\mathcal{A}_{auth} \subseteq (\mathcal{R}_{pfcg} \cup \{*\}) \times \mathcal{S} \times \mathcal{E} \times \mathcal{O}$ represents the set of coupled authorization capability tuples, with $\mathcal{S}_{auth}, \mathcal{E}_{auth}, \mathcal{O}_{auth}$ serving as marginal projections of $\mathcal{A}_{auth}$.

The Role Attenuation Engine enforces two strict filtering invariants before any prompt construction. Each tool $t \in \mathcal{T}$ is formally defined by its operational signature $t = \langle \text{id}_t, r_t, \text{srv}_t, \text{ent}_t, op_t, \text{Tags}(t) \rangle$, where r_t is the required PFCG role, srv_t is the target OData service, ent_t is the accessed entity set, $op_t \in \mathcal{O} = \{\text{READ}, \text{QUERY}, \text{CREATE}, \text{UPDATE}, \text{DELETE}, \text{EXECUTE}\}$ is the operational verb drawn from the universal operation domain, and $\text{Tags}(t)$ is the set of semantic domain tags:

1) **Fail-Closed Catalog Pruning:** To prevent Cartesian-product over-approximation across disjoint domain roles, each tool's required role r_t uniquely binds its operational capability $(\text{srv}_t, \text{ent}_t, op_t)$ in the catalog. A tool $t \in \mathcal{T}$ is retained in candidate set $\mathcal{T}_{\text{auth}}$ if and only if:

$$\begin{aligned} &\exists \tilde{r} \in ((\mathcal{R}_{\text{pfcg}} \cup \{*\}) \cap \{r_t, *\}) \\ &\text{s.t.} \quad (\tilde{r}, \text{srv}_t, \text{ent}_t, op_t) \in \mathcal{A}_{\text{auth}} \end{aligned} \quad (2)$$

Any tool violating this condition is evicted. Administrative endpoints (e.g., F110 automated payment disbursement or user role escalation) are eliminated from the search space. Stage 1 guarantees fail-closed containment across services, entity sets, and field properties; row-level organizational criteria (e.g., Company Code `BUKRS` or Plant `WERKS`) are enforced downstream via token propagation or parameter inspection.

2) **Column-Level Schema Masking:** For each tool t, let $\mathcal{M}_{\text{restr}}(t) \triangleq \{k \mid \text{ent}_t.k \in \mathcal{M}_{\text{restr}} \vee *.k \in \mathcal{M}_{\text{restr}}\}$ denote its entity-resolved attribute restriction projection. Partitioning input parameters into mandatory/key parameters and optional filter parameters, $\text{Params}(t) = \text{Params}_{\text{req}}(t) \cup \text{Params}_{\text{opt}}(t)$, if any mandatory input or primary-key lookup attribute is restricted $(\text{dom}(\text{Params}_{\text{req}}(t)) \cap \mathcal{M}_{\text{restr}}(t) \neq \emptyset)$, the Role Attenuation Engine fails closed by evicting the entire tool t from $\mathcal{T}_{\text{auth}}$, preventing downstream HTTP 400 schema validation failures. Otherwise, for each retained $t \in \mathcal{T}_{\text{auth}}$, $\text{Params}(t)$ and output schemas $\text{Out}(t)$ are projected to strip restricted attributes:

$$\begin{aligned} \text{Params}'(t) &= \{(k, v) \in \text{Params}(t) \mid k \notin \mathcal{M}_{\text{restr}}(t)\}, \\ \text{Out}'(t) &= \{(k, v) \in \text{Out}(t) \mid k \notin \mathcal{M}_{\text{restr}}(t)\} \end{aligned} \quad (3)$$

with $\text{Props}'(t) \triangleq \text{Params}'(t) \cup \text{Out}'(t)$ denoting the union of projected property sets. For analytical query tools and optional parameters, restricted fields (such as employee payroll masks `EmployeeSalaryMask` or confidential bank IBANs) are masked from property definitions and return schemas, preventing LLM awareness of confidential fields.

B. Just-In-Time Composite Tool Synthesis (Stage 2)

When an incoming query implies a multi-hop relational workflow—such as cross-entity reconciliation, master-detail verification, or multi-step transactional posting—executing the query via atomic tools requires multiple sequential LLM conversational round trips.

Agent-JIT incorporates an in-memory Abstract Syntax Tree (AST) workflow compiler that evaluates candidate workflows over authorized tools $\mathcal{T}_{\text{auth}}$. To bridge Stages 2 and 3 deterministically without circular dependencies: initial lexical-semantic scores $S_0(t, q)$ (Eqs. 9–10) and multigraph-amplified scores $S(t, q)$ (Eq. 11 with horizon $h_{\max} \in \{1, 2\}$) are evaluated first across $\mathcal{T}_{\text{auth}}$, and seed entities $\text{Seeds}(q)$ are selected as the top-$K_{\text{seed}} = 5$ entities ranked by $\max_{t \in \mathcal{T}_{\text{auth}}: e \in \text{Entities}(t)} S_0(t, q) > \theta$ (if only one seed exceeds θ under S_0, multigraph propagation extends to $h_{\max} = 2$ hops with geometric decay γ^h [$\gamma = 0.5, \gamma^2 = 0.25$] when no declarative template matches—or $h_{\max} = 1$ when a template matches—and appends at most $K_{\text{seed}} - 1 = 4$ top 1-hop and 2-hop entities promoted above θ by $S(t, q)$ to $\text{Seeds}(q)$ for an incremental BFS pass, strictly preserving $|\text{Seeds}(q)| \leq 5$ and bounding seed pairs to at most $\binom{5}{2} = 10$ while ensuring single-seed directed 2-hop chains $e_1 \to e_2 \to e_3$ promote both e_2 and e_3 into $\text{Seeds}(q)$). (i) *Declarative Template Triggering:* For structured enterprise workflows (such as AP 3-way matching or Order-to-Cash fulfillment), a pre-registered declarative template $\mathcal{W}$ is instantiated whenever at least two of its constituent entities belong to $\text{Seeds}(q)$, or when a single designated anchor entity belongs to $\text{Seeds}(q)$ and the query matches multi-step workflow intent tokens ($\text{IntentTokens}(\mathcal{W}) \cap \text{Tokens}(q) \neq \emptyset$, e.g., `reconcile`,

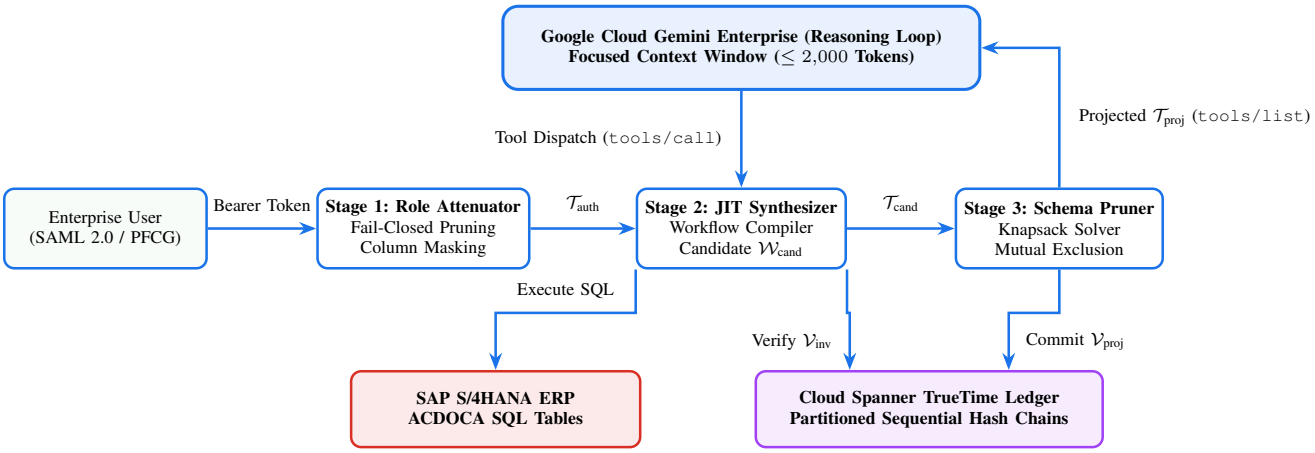


Fig. 1. Agent-JIT System Architecture: Intercepting enterprise agent reasoning loops in Google Cloud Gemini Enterprise to dynamically project role-attenuated tool schemas, compile candidate composite wrappers ($\mathcal{W}_{\text{cand}}$), solve knapsack sub-schema optimization with mutual exclusion, and commit linearizable TrueTime audit vouchers in Cloud Spanner without conflicting communication paths or box collisions.

`3-way`, `match`, `verify`, `fulfill`, enabling single-seed workflow queries such as "reconcile invoice INV-2026" to instantiate the 3-step template while preventing false-positive template triggering on purely single-entity atomic lookups); in Eq. (8), all constituent steps use the exact Stage 3 score $S(t_k, q)$ from Eq. 11 (so under template-matched $h_{\max} = 1$, a hop-2 constituent tool t_3 contributes $S(t_3, q) = S_0(t_3, q)$ while t_2 receives 1-hop amplification and the wrapper receives $\Delta_{\text{comp}} = 0.25$). (ii) *Dynamic BFS & Entity-to-Tool Inverse Mapping:* For ad-hoc read-only analytical queries where no declarative template matches, the synthesizer triggers dynamic BFS pathfinding over navigation edges E_N when $|\text{Seeds}(q)| \geq 2$, discovering minimal-hop relational paths ($m \leq 3$) connecting seed pairs with distinct constituent tools ($t_j \neq t_k$ for $j \neq k$, preventing cyclic self-joins) and coalescing strict 2-entity (1-hop) sub-paths into maximal $m = 3$-entity (2-hop) chains when all three entities belong to $\text{Seeds}(q)$. Each traversed entity e_k is mapped deterministically to an authorized read-only tool $\text{ToolID}_k \in \mathcal{T}_{\text{auth}}$ by selecting single-entity `READ` key-lookup tools for primary-key-bound hops and `QUERY` tools for collection filtering, breaking ties by highest $S_0(t, q)$ and then lexicographical id_t; if any intermediate entity e_k had all its `READ`/`QUERY` tools evicted in Stage 1, the path fails closed. (iii) *Semantic Disambiguation Across Equi-Length Foreign-Key Paths:* Property-level foreign-key bindings are extracted from OData CSDL [5] `<ReferentialConstraint>` metadata (`PrincipalRole` and `DependentRole`). When two seed entities are connected by multiple authorized minimal-hop paths with distinct business role semantics (e.g., `CreatedBy` vs. `ApprovedBy` or `BillTo` vs. `ShipTo`), all valid minimal paths are compiled into $\mathcal{W}_{\text{cand}}$ and disambiguated by a query-conditioned role-alignment term $\delta_{\text{fk}}(\mathcal{W}, q) = 0.05 \cdot |\text{RoleTokens}(\mathcal{W}) \cap \text{Tokens}(q)|$ added to $S(\mathcal{W}, q)$ in Eq. (8), where $\text{RoleTokens}(\mathcal{W})$ normalizes the CSDL `NavigationProperty` and `DependentRole` identifiers. Furthermore, Stage 2 constructs unified parameter and output schemas deterministically as $\text{Params}_{\text{unified}}(\mathcal{W}_{\text{comp}}) \triangleq \text{Prefix}_1(\text{Params}'(t_1)) \cup \bigcup_{k=2}^{m} \text{Prefix}_k(\text{Params}'(t_k) \setminus \text{BoundFK}(\sigma_k))$ and $\text{Out}'(\mathcal{W}_{\text{comp}}) \triangleq \bigcup_{k=1}^{m} \text{Prefix}_k(\text{Out}'(t_k))$—where $\text{Prefix}_k(\cdot)$ applies entity-qualified renaming `<Entity>_<Attr>` (or step-and-entity-qualified `S<k>_<Entity>_<Attr>` if distinct steps $j \neq k$ access the same entity set) prior to set union whenever unbound filter or output attribute names collide across constituent steps—strictly over Stage 1 pre-masked schemas, with unified composite restriction mask $\mathcal{M}_{\text{restr}}(\mathcal{W}_{\text{comp}}) \triangleq \bigcup_{k=1}^{m} \left(\mathcal{M}_{\text{restr}}(t_k) \cup \text{Prefix}_k(\mathcal{M}_{\text{restr}}(t_k))\right)$; if any required join key is masked on either endpoint ($k_{\text{fk}} \in \mathcal{M}_{\text{restr}}(t_{k-1}) \cup \mathcal{M}_{\text{restr}}(t_k)$), any required parameter of a compensating action $\text{Comp}(\sigma_k)$ lies in $\mathcal{M}_{\text{restr}}(\text{Comp}(t_k))$, or any forward output attribute in $\text{Out}(t_k)$ required to parameterize $\text{Comp}(\sigma_k)$ lies in $\mathcal{M}_{\text{restr}}(t_k)$, the engine prunes that candidate and fails closed.

To guarantee complete operational containment, a composite wrapper $\mathcal{W}_{\text{comp}} \in \mathcal{W}_{\text{cand}}$ is compiled if and only if all forward constituent steps and all mutating compensating actions reside within $\mathcal{T}_{\text{auth}}(\mathcal{P})$:

$$\mathcal{W}_{\text{comp}} = \langle \text{ID}_{\text{comp}}, \text{Name}, \text{Params}_{\text{unified}}, \vec{\sigma}, \mathcal{R}_{\cup}, \mathcal{H}_{\text{ast}} \rangle \quad (4)$$

where $\vec{\sigma} = \langle \sigma_1, \sigma_2, \ldots, \sigma_m \rangle$ is an ordered pipeline of atomic tool invocations with dynamic parameter bindings:

$$\sigma_k : \text{ToolID}_k(\text{Args}_k) \quad \text{where} \quad \text{Args}_k \leftarrow f(\text{Input}, \text{Output}_{<k}) \quad (5)$$

For 1:N collection cardinalities, $f(\text{Input}, \text{Output}_{<k})$ binds intermediate entity sets via OData `$expand` join pushdown for intra-service read queries, and disjunctive `$filter` set predicates or batch fan-out for cross-service steps. For each step σ_k in Equation (5), let $t_k \in \mathcal{T}_{\text{auth}}$ denote the invoked atomic tool and r_{σ_k} its required PFCG role in Equation (6), where $\mathcal{R}_{\cup}$ aggregates required roles across both forward and compensating operations:

$$\mathcal{R}_{\cup} \triangleq \bigcup_{k=1}^{m} \left(\{r_{\sigma_k}\} \cup \{r_{\text{Comp}(\sigma_k)} \mid \sigma_k \text{ is mutating}\} \right) \quad (6)$$

and $\mathcal{H}_{\text{ast}}$ is a deterministic SHA-256 fingerprint binding both forward and backward operational structures (with $\text{Comp}(\vec{\sigma}) \triangleq \langle \text{Comp}(\sigma_m), \ldots, \text{Comp}(\sigma_1) \rangle$, where $\text{Comp}(\sigma_k) = \text{null}$ for read-only steps):

$$\mathcal{H}_{\text{ast}} = \text{SHA-256}\Big(\text{CanonicalJSON}(\vec{\sigma}) \\ \| \text{CanonicalJSON}(\text{Comp}(\vec{\sigma})) \\ \| \text{Sort}(\mathcal{R}_{\cup}) \Big) \tag{7}$$

Compiled candidate composite wrappers form the set $\mathcal{W}_{\text{cand}}$, which expands the candidate pool for Stage 3 knapsack projection: $\mathcal{T}_{\text{cand}} \triangleq \mathcal{T}_{\text{auth}} \cup \mathcal{W}_{\text{cand}}$. For each composite wrapper $\mathcal{W} \in \mathcal{W}_{\text{cand}}$, its relevance utility is defined super-additively over its forward constituent steps $\text{Constituents}_{\text{fwd}}(\mathcal{W}) \triangleq \{t_k \mid k \in \{1, \ldots, m\}\}$:

$$S(\mathcal{W}, q) \triangleq \sum_{t \in \text{Constituents}_{\text{fwd}}(\mathcal{W})} S(t, q) + \Delta_{\text{comp}} + \delta_{\text{fk}}(\mathcal{W}, q) \tag{8}$$

where $S(t, q)$ is the multigraph-amplified relevance score formally defined in Stage 3 (Eq. 11) and $\Delta_{\text{comp}} = 0.25$ is the composition utility bonus, guaranteeing that admitting $\mathcal{W}$ yields higher objective utility than individual constituent tools when feasible under budget B.

When the LLM invokes $\mathcal{W}_{\text{comp}}$, the Agent-JIT runtime executes the constituent OData operations sequentially within the server process against physical database tables, passing intermediate keys without returning to the model between steps.

Transactional Failure Semantics & Non-Compensable Boundaries: In multi-step mutating workflows, single-endpoint transaction batches execute within native SAP OData $batch changesets, rolling back atomically via `ROLLBACK WORK` on failure. For multi-request distributed pipelines, the execution engine coordinates backward compensating operations following classical Saga protocols [35], [36], executing pre-registered inverse actions $\text{Comp}(\sigma_k)$ (e.g., reversing journal postings or releasing reservation locks) to restore backend consistency; to prevent dirty reads across concurrent MCP sessions during the < 15 ms fault-recovery window prior to $\text{Comp}(\sigma_1)$, intermediate steps tag records with a provisional state (`Status = PROVISIONAL_PENDING`) that is excluded by default in Agent-JIT read projections until the final step commits. Conversely, for read-only reconciliation queries (such as AP 3-way matching), the pipeline performs relational join pushdown across foreign-key navigation paths, aggregating results in a single turn without intermediate LLM round trips and without requiring compensating database mutations. For non-compensable actions with irreversible real-world physical side effects (such as settled external SWIFT/Fedwire wire transfers), the engine validates all input preconditions and suspends automated execution prior to dispatching σ_1 (avoiding uncommitted mid-pipeline locks during human approval waits), returning an intermediate task token τ_{task} with status `PENDING_AUTHORIZATION` and requiring out-of-band dual-custody human sign-off before execution.

C. Goal-Conditioned Sub-Schema Pruning (Stage 3)

Following role attenuation and candidate composite wrapper compilation, the candidate pool $\mathcal{T}_{\text{cand}} = \mathcal{T}_{\text{auth}} \cup \mathcal{W}_{\text{cand}}$ may contain hundreds of authorized atomic and composite tools. The Schema Pruner selects the most goal-relevant tools subject to a token budget constraint B (default $B = 2{,}000$ tokens).

Definition 2 (Heterogeneous Capability Multigraph)**.** *Let $G = (V_T \cup V_E, E_R \cup E_N)$ be a heterogeneous capability multigraph where $V_T \equiv \mathcal{T}_{auth}$ is the set of authorized tools, V_E is the set of OData entity sets, $E_R \subseteq V_T \times V_E$ represents bipartite tool-entity operational bindings, and $E_N \subseteq V_E \times V_E$ represents directional navigation associations (foreign key relationships) between entities.*

In compliance with the Model Context Protocol (MCP) JSON-RPC 2.0 specification, caller natural language goal q and identity assertions are conveyed via standard protocol parameter extensions (under `params._meta.user_goal` or session transport headers `X-Agent-Goal`). For goal q, the engine computes an initial semantic relevance score $S_0(t, q)$ combining dampened maximum-scaled BM25 lexical token matching and continuous dense semantic embeddings $\mathbf{e}_q, \mathbf{e}_t \in \mathbb{R}^d$:

$$S_0(t, q) = \beta \cdot \widetilde{\text{BM25}}(q, \text{Doc}(t)) \\ + (1 - \beta) \cdot \max(0, \cos(\mathbf{e}_q, \mathbf{e}_t)) \\ + \alpha \cdot \mathbf{1}_{\{\text{Tags}(t) \cap \text{Tokens}(q) \neq \emptyset\}} \tag{9}$$

where $\text{Doc}(t)$ concatenates tool name, description, and strictly the Stage 1 pre-masked schema properties $\text{Props}'(t)$ (excluding restricted attributes in $\mathcal{M}_{\text{restr}}(t)$ so masked fields never influence ranking), $\text{Tokens}(q)$ denotes lowercased alphanumeric query tokens after stopword removal matching the vocabulary of $\text{Tags}(t)$, $\beta = 0.5$ balances lexical and semantic density, $\alpha = 0.35$ is the semantic category bonus weight, $\text{BM25}(q, \text{Doc}(t)) \in [0, 1]$ denotes raw Okapi BM25 ($k_1 = 1.5, b = 0.75$, using global corpus IDF and avgdl precomputed offline across $\mathcal{T}$) pre-normalized by the query's theoretical upper bound $Z_{\text{BM25}}(q) \triangleq \max\left(2 \cdot \text{IDF}_{\text{med}}, \sum_{w \in \text{Tokens}(q)} \text{IDF}(w) \cdot (k_1 + 1)\right)$ (where $2 \cdot \text{IDF}_{\text{med}}$ is a two-term median-IDF floor ensuring an isolated single-token hit cannot self-normalize to 1.0), and dampened catalog-max scaling with soft saturation constant $k_{\text{sat}} = 0.50$ and zero-clamping is defined as:

$$\widetilde{\text{BM25}}(q, \text{Doc}(t)) \triangleq \mathbf{1}_{\{\text{BM25}(q, \text{Doc}(t)) \geq \tau_{\text{floor}}\}} \\ \cdot \max\left(0, \frac{\text{BM25}(q, \text{Doc}(t))}{\max_{t' \in \mathcal{T}_{\text{auth}}} \text{BM25}(q, \text{Doc}(t')) + k_{\text{sat}} + \epsilon}\right) \tag{10}$$

with calibrated normalized score floor $\tau_{\text{floor}} = 0.10$, soft saturation constant $k_{\text{sat}} = 0.50$ (which eliminates step discontinuities when $\max_{t'} \text{BM25}(q, \text{Doc}(t')) \approx \tau_{\text{floor}}$, damping an isolated single-token lexical hit to ≤ 0.17 so it cannot unilaterally exceed $\theta = 0.40$ without semantic or tag corroboration, while multi-token lexical matches with BM25 $\gg \tau_{\text{floor}}$ approach $\frac{1}{1+k_{\text{sat}}} \approx 0.67$), and $\epsilon = 10^{-6}$, ensuring $\widetilde{\text{BM25}}(q, \text{Doc}(t)) \in [0, 1)$ and $S_0(t, q) \in [0, 1.35]$, with $\theta = 0.40$ acting as a calibrated propagation filter.

To account for relational entity context across multiple upstream dependencies, relevance scores are propagated across the capability multigraph G up to horizon $h_{\max} \in \{1, 2\}$ hops with geometric decay γ^h ($\gamma = 0.5$). For any tool t with $S_0(t, q) > \theta$, the engine traverses navigation edges E_N up to $h \in \{1, \ldots, h_{\max}\}$ hops (where $h_{\max} = 1$ in standard multi-seed or template-matched composite mode, and $h_{\max} = 2$ on single-seed ad-hoc queries without template matches as well as in atomic-only mode where Stage 2 is disabled):

$$S(t', q) = S_0(t', q) + \max_{h \in \{1, \ldots, h_{\max}\}} \gamma^h \cdot \max\left(0, \max_{\substack{t \in \mathcal{T}_{\text{auth}} \\ \text{Entities}(t) \cap \text{Adj}_h(\text{Entities}(t')) \neq \emptyset \\ S_0(t,q) > \theta}} S_0(t, q)\right) \quad (11)$$

where $\text{Entities}(t) \triangleq \{e \in V_E \mid (t, e) \in E_R\}$ denotes entity sets operated on by tool t, $\text{Adj}_h(\mathcal{E})$ represents the exact h-hop shortest-path relational neighborhood of entity set $\mathcal{E}$ over undirected navigation edges E_N ($\text{Adj}_1(\mathcal{E}) \equiv \bigcup_{e \in \mathcal{E}} \{e' \in V_E \mid (e, e') \in E_N \vee (e', e) \in E_N\}$), and $\gamma = 0.5$ is the graph attenuation decay parameter ($\gamma^1 = 0.5, \gamma^2 = 0.25$). If no qualifying tool exists, the inner maximization defaults to 0.

The engine then solves the 0/1 knapsack projection problem [42] over $\mathcal{T}_{\text{cand}}$ augmented with set-packing mutual-exclusion constraints:

$$\max_{\mathbf{x}} \quad \sum_{c \in \mathcal{T}_{\text{cand}}} x_c \cdot S(c, q) \quad (12)$$

$$\text{s.t.} \quad \sum_{c \in \mathcal{T}_{\text{cand}}} x_c \cdot \text{Cost}(c) \leq B, \quad x_c \in \{0, 1\} \quad (13)$$

$$x_t + \sum_{\mathcal{W} \in \mathcal{W}_{\text{cand}} : t \in \text{Constituents}_{\text{fwd}}(\mathcal{W})} x_{\mathcal{W}} \leq 1 \quad \forall t \in \mathcal{T}_{\text{auth}} \quad (14)$$

where $c \in \mathcal{T}_{\text{cand}} = \mathcal{T}_{\text{auth}} \cup \mathcal{W}_{\text{cand}}$ indexes arbitrary candidate items (preventing symbol overloading with atomic tools $t \in \mathcal{T}_{\text{auth}}$ in Constraint 14). To maintain sub-millisecond in-process latency without running a full BPE subword tokenizer online, $\text{Cost}(c) \geq 1$ is computed in $O(1)$ time per candidate using pre-tabulated per-tool unmasked token sums $\text{Cost}_{\text{params}}(t) \triangleq \sum_{k \in \text{dom}(\text{Params}'(t))} \text{tok}(k)$ and $\text{Cost}_{\text{out}}(t) \triangleq \sum_{k \in \text{dom}(\text{Out}'(t))} \text{tok}(k)$ alongside pre-tabulated collision-prefix token deltas $\Delta\text{tok}_{\text{prefix}}(\mathcal{W})$ (summing the pre-tabulated BPE token delta $\text{tok}(\text{Prefix}_k(a)) - \text{tok}(a)$ strictly over colliding attributes minus bound foreign keys $\sum_{k \in \text{BoundFK}(\sigma_k)} \text{tok}(k)$): $\text{Cost}(t) = \text{Cost}_{\text{base}}(t) + \text{Cost}_{\text{params}}(t) + \text{Cost}_{\text{out}}(t)$, and $\text{Cost}(\mathcal{W}) = c_{\text{hdr}} + \sum_{k=1}^{m} \left(\text{Cost}_{\text{hdr}}(t_k) + \text{Cost}_{\text{params}}(t_k) + \text{Cost}_{\text{out}}(t_k)\right) + \Delta\text{tok}_{\text{prefix}}(\mathcal{W})$. Constraint (14) enforces set-packing mutual exclusion: if a composite wrapper $\mathcal{W}$ is selected ($x_{\mathcal{W}} = 1$), its constituent atomic tools are suppressed ($x_t = 0$), and no two overlapping composite wrappers sharing tool t can be co-admitted.

The greedy solver processes $c \in \mathcal{T}_{\text{cand}}$ in descending order of value density $\frac{S(c,q)}{\text{Cost}(c)}$ (breaking ties deterministically by minimal $\text{Cost}(c)$, then ascending lexicographical ID):

1) **Forward Blocking:** When a composite wrapper $\mathcal{W}$ is evaluated with $\text{AdmittedConstituents}(\mathcal{W}) \triangleq \text{Constituents}_{\text{fwd}}(\mathcal{W}) \cap \mathcal{T}_{\text{proj}} = \emptyset$ and disjoint from the forward-blocked constituent sets of all already-admitted wrappers, $\mathcal{W}$ is admitted if $\text{Cost}(\mathcal{W})$ fits the remaining budget, and all $t \in \text{Constituents}_{\text{fwd}}(\mathcal{W})$ are marked as forward-blocked for subsequent iterations. Any atomic tool t in the forward-blocked set is skipped, whereas any composite wrapper $\mathcal{W}$ intersecting already-admitted constituents or wrappers is evaluated under Backward Replacement below.
2) **Backward Replacement:** For any candidate wrapper $\mathcal{W}$ whose forward constituents intersect the currently admitted projection, let its conflicting admitted set be $\text{Evicted}(\mathcal{W}) \triangleq (\text{Constituents}_{\text{fwd}}(\mathcal{W}) \cap \mathcal{T}_{\text{proj}}) \cup \{\mathcal{W}' \in \mathcal{T}_{\text{proj}} \cap \mathcal{W}_{\text{cand}} \mid \text{Constituents}_{\text{fwd}}(\mathcal{W}') \cap \text{Constituents}_{\text{fwd}}(\mathcal{W}) \neq \emptyset\}$. When $\text{Evicted}(\mathcal{W}) \neq \emptyset$ and either $|\text{Constituents}_{\text{fwd}}(\mathcal{W}) \cap \mathcal{T}_{\text{proj}}| \geq 2$, $\text{Evicted}(\mathcal{W})$ contains any conflicting admitted composite wrapper $\mathcal{W}' \in \mathcal{T}_{\text{proj}} \cap \mathcal{W}_{\text{cand}}$ (including strict sub-wrappers and equi-length sibling wrappers), or $\text{IntentTokens}(\mathcal{W}) \cap \text{Tokens}(q) \neq \emptyset$ (preventing a multi-step wrapper from evicting a single atomic tool on purely single-entity queries; if a wrapper $\mathcal{W}$ is deferred when $|\text{Constituents}_{\text{fwd}}(\mathcal{W}) \cap \mathcal{T}_{\text{proj}}| = 1$, it is registered in a watcher list and immediately re-evaluated if a second constituent $t_2 \in \text{Constituents}_{\text{fwd}}(\mathcal{W})$ is subsequently admitted during the pass), $\mathcal{W}$ evicts $\text{Evicted}(\mathcal{W})$ and replaces it whenever net utility increases ($\Delta S = S(\mathcal{W}, q) - \sum_{c' \in \text{Evicted}(\mathcal{W})} S(c', q) > 0$) and net cost satisfies $\Phi(\mathcal{T}_{\text{proj}}) + \Delta C \leq B$, where $\Delta C = \text{Cost}(\mathcal{W}) - \sum_{c' \in \text{Evicted}(\mathcal{W})} \text{Cost}(c')$. Upon replacement, the forward-blocked set is updated to reflect all remaining admitted wrappers. While classical weight-ordered unguarded local exchange over m-set packing ($m \leq 3$) provides a worst-case $\frac{1}{m}$-approximation when the knapsack budget is non-binding, our single-pass value-density-ordered ($\frac{S(c,q)}{\text{Cost}(c)}$) knapsack heuristic with intent-guarded replacement explicitly optimizes token efficiency under budget B in $O(|\mathcal{T}_{\text{cand}}| \log |\mathcal{T}_{\text{cand}}|)$ time and empirically matches the exact Integer Linear Programming (ILP) optimum on 99.6% of our 850 benchmark queries.
3) **Liveness Override:** If no candidate tool fits within budget B (which would otherwise leave $\mathcal{T}_{\text{proj}} = \emptyset$), the solver admits the single highest value-density candidate $c^* \triangleq \arg\max_{c \in \mathcal{T}_{\text{cand}}} \frac{S(c,q)}{\text{Cost}(c)}$ (ties broken by minimal $\text{Cost}(c)$, then lexicographical ID) as a singleton fallback, ensuring $\Phi(\mathcal{T}_{\text{proj}}) \leq \max(B, \text{Cost}(c^*))$.

D. Cloud Spanner Linearizable TrueTime Auditing (Stage 4)

To enforce zero-trust execution integrity in regulated enterprise environments, Agent-JIT integrates directly with Google Cloud Spanner [6]. To decouple the asynchronous lifecycle between prompt sub-schema negotiation and downstream tool dispatch without symbol overloading or uncommitted state, Agent-JIT provisions two linked records:

Definition 3 (Two-Stage Linearizable Audit Vouchers)**.** *The zero-trust audit lifecycle comprises two cryptographically linked records:*

1) ***Projection Voucher*** ($\mathcal{V}_{proj}$)**:** *Committed during* `tools/list` *negotiation prior to LLM reasoning:*

$$\mathcal{V}_{proj} = \langle v_{id}, \tau, s, u, u_{sap}, h_q, h_{role}, \mathcal{D}_{proj}, t_{req}, \Sigma_{proj} \rangle \quad (15)$$

where $v_{id} \in \{0,1\}^{128}$ *is a UUIDv4 transaction identifier,* τ *is the enterprise tenant ID,* s *is the agent conversation/session identifier,* u *is the authenticated caller identity,* u_{sap} *is the impersonated backend SAP service user,* $h_q = \text{SHA-256}(q)$ *is the request goal digest,* $h_{role} = \text{SHA-256}(\text{Sort}(\mathcal{R}_{pfcg}) \parallel \text{Sort}(\mathcal{M}_{restr}) \parallel \text{Sort}(\mathcal{A}_{auth}))$ *is the canonically sorted role attenuation digest binding the principal's capability matrix* $\mathcal{A}_{auth}$*,* $\mathcal{D}_{proj} \triangleq \{(c_{id}, \mathcal{H}_{ast}(c)) \mid c \in \mathcal{T}_{proj}\}$ *commits paired descriptor tuples (equivalently indexed as a finite map* $\mathcal{D}_{proj}[c_{id}]$*) for each admitted candidate* $c \in \mathcal{T}_{proj}$ *with identifier* $c_{id} \in \{id_t, ID_{comp}\}$ *(where* $\mathcal{H}_{ast}(t) = null$ *for atomic tools and* $\text{IDs}(\mathcal{D}_{proj}) \triangleq \{c_{id} \mid (c_{id}, \cdot) \in \mathcal{D}_{proj}\}$*; for composite wrappers, the compiled execution plan* $(\vec{\sigma}, \text{Comp}(\vec{\sigma}), \mathcal{R}_{\cup})$ *is serialized alongside* $\mathcal{V}_{proj}$ *in the shared Redis ring buffer keyed by* (v_{id}, c_{id}) *so stateless failover replicas can retrieve and verify it),* t_{req} *is a local monotonic request arrival timestamp, and* Σ_{proj} *is the cryptographic integrity commitment:*

$$\Sigma_{proj} = \text{SHA-256}\Big(v_{id} \parallel \tau \parallel s \parallel u \parallel u_{sap} \parallel h_q \parallel h_{role} \parallel \text{Sort}(\mathcal{D}_{proj}) \parallel t_{req}\Big) \quad (16)$$

2) ***Invocation Voucher*** ($\mathcal{V}_{inv}$)**:** *Committed during* `tools/call` *execution dispatch:*

$$\mathcal{V}_{inv} = \langle v_{id}, ToolID_{inv}, \mathcal{H}^{exec}_{ast}, h_{args}, t_{exec}, \Sigma_{inv} \rangle \quad (17)$$

where $ToolID_{inv}$ *is the invoked tool identifier,* $\mathcal{H}^{exec}_{ast}$ *is the dispatch-time AST digest recomputed via Eq. (7) over the retrieved execution plan* $(\vec{\sigma}, \text{Comp}(\vec{\sigma}), \mathcal{R}_{\cup})$ *(or null for atomic tools),* $h_{args} = \text{SHA-256}(\text{CanonicalJSON}(\langle Args, \{Args_k\}_{k=1}^{m} \rangle))$ *commits both top-level wire arguments Args and resolved step arguments* $Args_k$ *(Eq. 5),* t_{exec} *is the dispatch timestamp, and* Σ_{inv} *explicitly binds the recomputed AST digest and parameters to* Σ_{proj}*:*

$$\Sigma_{inv} = \text{SHA-256}\left(\Sigma_{proj} \parallel ToolID_{inv} \parallel \mathcal{H}^{exec}_{ast} \parallel h_{args} \parallel t_{exec}\right) \quad (18)$$

Furthermore, audit vouchers are linked into partitioned linearizable cryptographic hash chains advancing across both projection and execution stages:

$$\mathcal{C}^{(\tau,s)}_{i,\text{proj}} = \text{SHA-256}\left(\mathcal{C}^{(\tau,s)}_{i-1,\text{term}} \parallel v^{(i)}_{\text{id}} \parallel t^{(i)}_{\text{req}} \parallel \Sigma^{(i)}_{\text{proj}}\right) \quad (19)$$

$$\mathcal{C}^{(\tau,s)}_{i,\text{inv}} = \text{SHA-256}\left(\mathcal{C}^{(\tau,s)}_{i,\text{head}} \parallel \text{ToolID}^{(i)}_{\text{inv}} \parallel t^{(i)}_{\text{exec}} \parallel \Sigma^{(i)}_{\text{inv}}\right) \quad (20)$$

where $\mathcal{C}^{(\tau,s)}_{0,\text{term}} \triangleq \text{SHA-256}(\tau \parallel s \parallel \text{“GENESIS”} \parallel t_{\text{init}})$ initializes the session chain at $i = 1$, $\mathcal{C}^{(\tau,s)}_{i-1,\text{term}}$ denotes the terminal hash of the prior conversational turn, and the current turn's terminal hash is defined as $\mathcal{C}^{(\tau,s)}_{i,\text{term}} \in \{\mathcal{C}^{(\tau,s)}_{i,\text{inv}}, \mathcal{C}^{(\tau,s)}_{i,\text{conv}}, \mathcal{C}^{(\tau,s)}_{i,\text{rej}}, \mathcal{C}^{(\tau,s)}_{i,\text{susp}}, \mathcal{C}^{(\tau,s)}_{i,\text{timeout}}\}$. Each turn i maintains an intra-turn head pointer $\mathcal{C}^{(\tau,s)}_{i,\text{head}}$ initialized to $\mathcal{C}^{(\tau,s)}_{i,\text{proj}}$, and Eq. (20) chains from the current $\mathcal{C}^{(\tau,s)}_{i,\text{head}}$ to prevent intra-turn branch forking: (i) if the LLM produces a conversational response without invoking tools, the chain advances via $\mathcal{C}^{(\tau,s)}_{i,\text{conv}} = \text{SHA-256}(\mathcal{C}^{(\tau,s)}_{i,\text{head}} \parallel v^{(i)}_{\text{id}} \parallel \text{null} \parallel t^{(i)}_{\text{resp}})$ committed via host completion callbacks (or if the client crashes/times out after $\Delta_{\text{turn}} = 60$ s without invoking a tool, the gateway seals turn i with $\mathcal{C}^{(\tau,s)}_{i,\text{timeout}} = \text{SHA-256}(\mathcal{C}^{(\tau,s)}_{i,\text{head}} \parallel v^{(i)}_{\text{id}} \parallel \text{“TIMEOUT”} \parallel t^{(i)}_{\text{exp}})$ before admitting turn $i+1$); (ii) if an unauthorized invocation is blocked within an active turn i, the chain updates $\mathcal{C}^{(\tau,s)}_{i,\text{head}} \leftarrow \mathcal{C}^{(\tau,s)}_{i,\text{rej}} = \text{SHA-256}(\mathcal{C}^{(\tau,s)}_{i,\text{head}} \parallel \text{ToolID}^{(i)}_{\text{blocked}} \parallel -32001 \parallel t^{(i)}_{\text{rej}})$ (where $\text{ToolID}^{(i)}_{\text{blocked}}$ is the comma-joined lexicographically sorted list of failed tool IDs) without invalidating the open $\mathcal{V}^{(i)}_{\text{proj}}$ so any subsequent valid retry in turn i chains linearly from $\mathcal{C}^{(\tau,s)}_{i,\text{head}}$ (and if turn i aborts or terminates following the rejection without a retry, the turn's terminal hash seals directly as $\mathcal{C}^{(\tau,s)}_{i,\text{term}} \leftarrow \mathcal{C}^{(\tau,s)}_{i,\text{head}} = \mathcal{C}^{(\tau,s)}_{i,\text{rej}}$), whereas unsolicited socket probes lacking a valid v_{id} are appended to an isolated anomaly sub-chain $\mathcal{C}^{(\tau,s)}_{i,\text{probe}} = \text{SHA-256}(\mathcal{C}^{(\tau,s)}_{i-1,\text{term}} \parallel \text{ToolID}^{(i)}_{\text{blocked}} \parallel -32001 \parallel t^{(i)}_{\text{rej}})$ that never mutates $\mathcal{C}^{(\tau,s)}_{i-1,\text{term}}$ or desynchronizes an active in-flight turn i; (iii) if a non-compensable operation suspends for human sign-off, turn i synchronously flushes $\mathcal{V}^{(i)}_{\text{proj}}$ and anchors with $\mathcal{C}^{(\tau,s)}_{i,\text{susp}} = \text{SHA-256}(\mathcal{C}^{(\tau,s)}_{i,\text{head}} \parallel \text{ToolID}^{(i)}_{\text{inv}} \parallel h^{(i)}_{\text{args}} \parallel \tau_{\text{task}} \parallel \text{“PENDING_AUTHORIZATION”} \parallel t^{(i)}_{\text{susp}})$ in Cloud Spanner, locking mutating turns on session (τ, s) (or if read-only turns $i{+}1, \ldots, k{-}1$ intervene before approval arrives at turn $k \geq i+1$, resume turn k dispatches directly via τ_{task}, resolves $\mathcal{V}^{(i)}_{\text{proj}}$ from Spanner, sets $\mathcal{C}^{(\tau,s)}_{k,\text{head}} = \text{SHA-256}(\mathcal{C}^{(\tau,s)}_{k-1,\text{term}} \parallel \Sigma_{\text{auth}})$—or chains from $\mathcal{C}^{(\tau,s)}_{k,\text{proj}}$ if a turn-k `tools/list` preceded resumption—computes $\Sigma^{(k)}_{\text{inv}}$ binding both original $\Sigma^{(i)}_{\text{proj}}$ and Σ_{auth}, and re-verifies strict role digest equality $h^{\text{live}}_{\text{role}} = \mathcal{V}^{(i)}_{\text{proj}}.h_{\text{role}}$ in Check (iii), AST integrity Check (v), and approver sign-off role freshness at $t^{(k)}_{\text{exec}}$ before evaluating $\mathcal{C}^{(\tau,s)}_{k,\text{inv}}$); and (iv) if k_{par} parallel tool calls are issued within a turn, individual invocations $j \in \{1, \ldots, k_{\text{par}}\}$ map to Merkle leaves $\text{leaf}_j = \text{SHA-256}(j \parallel \text{ToolID}^{(i,j)}_{\text{inv}} \parallel \mathcal{H}^{\text{exec},(i,j)}_{\text{ast}} \parallel h^{(i,j)}_{\text{args}} \parallel t^{(i,j)}_{\text{exec}})$ sorted canonically by $(\text{ToolID}^{(i,j)}_{\text{inv}}, h^{(i,j)}_{\text{args}}, j)$—using arrival index j as a deterministic tie-breaker when identical tools and arguments are invoked in parallel—and aggregated into a deterministic Merkle sub-tree root $R^{(i)}_{\text{Merkle}}$ before computing turn commitment $\Sigma^{(i)}_{\text{inv}} = \text{SHA-256}(\Sigma^{(i)}_{\text{proj}} \parallel R^{(i)}_{\text{Merkle}} \parallel t^{(i)}_{\text{exec}})$ and chaining $\mathcal{C}^{(\tau,s)}_{i,\text{inv}} = \text{SHA-256}(\mathcal{C}^{(\tau,s)}_{i,\text{head}} \parallel R^{(i)}_{\text{Merkle}} \parallel t^{(i)}_{\text{exec}} \parallel \Sigma^{(i)}_{\text{inv}})$, where parallel tool batches validate all k_{par} invocations atomically prior to execution so that any single violation transitions the entire turn fail-closed to $\mathcal{C}^{(\tau,s)}_{i,\text{rej}}$. During `tools/call`, the client runtime propagates v_{id} via standard MCP metadata extensions (`params._meta.voucher_id`) or HTTP session headers (`X-Agent-Voucher-ID`).

Partitioning hash chains per tenant τ and session s prevents row-lock contention across distributed Spanner splits at high throughput, while cross-session linearizability is guaranteed by Spanner's hardware TrueTime intervals $[t_{\min}, t_{\max}]$ and multi-Paxos commit consensus [6], [37]. In our physical testbed, the ledger is evaluated via a persistent SQLite database in WAL mode (`spanner_truetime_ledger.db`) executing on physical disk. In distributed production deployments, zero-trust gating validates synchronously against an in-memory commit ring buffer (synchronized across container instances via Google Cloud Memorystore for Redis with append-only persistence) within sub-millisecond bounds, while append-only Spanner transactions flush durability writes asynchronously in the background (or synchronously in high-security compliance mode).

Zero-Trust Containment Verification: When an LLM runtime attempts to invoke tool $\text{ToolID}_{\text{inv}}$ (denoting the corresponding tool object as t_{inv}), the execution engine validates: (i) tool membership $\text{ToolID}_{\text{inv}} \in \text{IDs}(\mathcal{V}_{\text{proj}}.\mathcal{D}_{\text{proj}})$, (ii) session/principal binding $(u, \tau, s, h_{\text{role}}) = (\mathcal{V}_{\text{proj}}.u, \mathcal{V}_{\text{proj}}.\tau, \mathcal{V}_{\text{proj}}.s, \mathcal{V}_{\text{proj}}.h_{\text{role}})$, (iii) role freshness confirming h_{role} has not been invalidated by dynamic SCIM (RFC 7644 [41]) hooks, (iv) parameter containment $\big(\text{dom}(\text{Args}) \cup \bigcup_{k=1}^{m} \text{dom}(\text{Args}_k)\big) \cap \mathcal{M}_{\text{restr}}(t_{\text{inv}}) = \emptyset$ across both top-level and internally bound step arguments while restricting backend queries strictly to $\text{dom}(\text{Out}'(t_{\text{inv}}))$, and (v) if $\text{ToolID}_{\text{inv}}$ is a composite wrapper, that its dispatch-time recomputed AST digest $\mathcal{H}_{\text{ast}}^{\text{exec}}$ (Eq. 7 evaluated over the execution plan retrieved from the shared Redis ring buffer for $(v_{\text{id}}, \text{ToolID}_{\text{inv}})$) equals the committed digest $\mathcal{V}_{\text{proj}}.\mathcal{D}_{\text{proj}}[\text{ToolID}_{\text{inv}}]$. Any violation is rejected fail-closed with status `SECURITY_VIOLATION` and MCP error code -32001.

Dynamic Role Cache Invalidation: To accommodate real-time authorization changes in SAP PFCG or enterprise IdPs, the engine caches role attenuation states in Redis via `HSET` keyed by tenant-scoped principal tuple (τ, u) with hash field h_{role}. Invalidation hooks listening to SAP Event Mesh or IdP SCIM webhooks evict the principal's Redis hash key (τ, u) in $O(1)$ time upon role modifications without needing prior knowledge of h_{role}; on any Redis cache miss during Check (iii), the gateway re-fetches live role bindings from the enterprise IdP/SAP authority and recomputes h_{role} to distinguish a benign cold cache miss from an active SCIM revocation, guaranteeing immediate fail-closed policy convergence.

III. Theoretical Guarantees

Lemma 1 (Prompt Perturbation Invariance of Role Attenuation)**.** *Let $q' = q \oplus \delta$ be an arbitrary adversarial prompt perturbation where δ contains prompt-injection tokens attempting privilege escalation. Because Stage 1 principal-propagated role attenuation evaluates caller identity $\mathcal{P}$ strictly against backend authorization matrices prior to and independent of prompt text, candidate set selection is structurally orthogonal to the prompt input channel ($\mathcal{T}_{auth} \perp q$), guaranteeing:*

$$\mathcal{T}_{auth}(\mathcal{P}, q \oplus \delta) = \mathcal{T}_{auth}(\mathcal{P}) \tag{21}$$

Consequently, while prompt perturbation δ can influence ranking within $\mathcal{T}_{auth}$ during Stage 3 sub-schema pruning, it is mathematically incapable of expanding the atomic candidate set beyond $\mathcal{T}_{auth}(\mathcal{P})$, thereby strictly bounding the downstream reachable capability closure $\mathcal{B}(\mathcal{P})$.

Theorem 1 (Fail-Closed Security and Column-Level Containment)**.** *Let $\mathcal{P}$ be an authenticated principal with authorized capability boundary defined constructively as:*

$$\mathcal{B}(\mathcal{P}) \triangleq \mathcal{T}_{auth}(\mathcal{P}) \cup \Big\{ \mathcal{W}_{comp}(\vec{\sigma}) \;\Big|\; \forall \sigma_k \in \vec{\sigma},\ t_k \in \mathcal{T}_{auth}(\mathcal{P}), \ \text{Comp}(t_k) \in \mathcal{T}_{auth}(\mathcal{P}) \ \textit{if}\ \text{IsMutating}(t_k) \Big\} \tag{22}$$

For any prompt query q and any adversarial prompt perturbation δ, admitted wire-level MCP tool invocations $\mathcal{T}_{admitted}^{MCP}$ and backend physical operations $\mathcal{T}_{ops}^{backend}$ satisfy:

$$\mathcal{T}_{admitted}^{MCP} \subseteq \mathcal{T}_{proj} \subseteq \mathcal{B}(\mathcal{P}) \quad \textit{and} \quad \mathcal{T}_{ops}^{backend} \subseteq \bigcup_{t \in \mathcal{T}_{admitted}^{MCP}} \text{Constituents}_{ops}(t) \subseteq \mathcal{T}_{auth}(\mathcal{P}) \subseteq \mathcal{B}(\mathcal{P}) \tag{23}$$

where $\text{Constituents}_{ops}(t) = \{t\}$ for atomic tools $t \in \mathcal{T}_{auth}$, and for composite wrappers $\mathcal{W}_{comp}$, the full operational closure is defined as:

$$\text{Constituents}_{ops}(\mathcal{W}_{comp}) \triangleq \text{Constituents}_{fwd}(\mathcal{W}_{comp}) \cup \big\{ \text{Comp}(t_k) \mid k \in \{1, \dots, m\} \wedge \text{IsMutating}(t_k) \big\} \tag{24}$$

which strictly conditions compensating handlers on mutability and guarantees complete operational containment and type homogeneity in $\mathcal{T}_{auth}(\mathcal{P})$. Any incoming call $\text{ToolID}_{inv} \notin \text{IDs}(\mathcal{V}_{proj}.\mathcal{D}_{proj})$ is rejected fail-closed with error -32001. Furthermore, column-level schema masking is strictly invariant:

$$\forall t \in \mathcal{T}_{proj}, \quad \big(\text{dom}(\textit{Params}'(t)) \cup \text{dom}(\textit{Out}'(t))\big) \cap \mathcal{M}_{restr}(t) = \emptyset \tag{25}$$

where $\text{dom}(\textit{Params}'(t)) \triangleq \{k \mid (k, v) \in \textit{Params}'(t)\}$ and $\text{dom}(\textit{Out}'(t)) \triangleq \{k \mid (k, v) \in \textit{Out}'(t)\}$ (with $\textit{Params}'(\mathcal{W}_{comp}) \equiv \textit{Params}_{unified}(\mathcal{W}_{comp})$ for composite wrappers).

Proof. By Lemma 1, Stage 1 role pruning is strictly invariant to δ, ensuring $\mathcal{T}_{\text{auth}}(\mathcal{P}, q \oplus \delta) = \mathcal{T}_{\text{auth}}(\mathcal{P}) \subseteq \mathcal{B}(\mathcal{P})$. In Stage 2, candidate composite wrappers $\mathcal{W}_{\text{cand}}$ are synthesized exclusively when all constituent forward steps and mutating compensating actions satisfy $t_k, \text{Comp}(t_k) \in \mathcal{T}_{\text{auth}}$, which by constructive definition yields $\mathcal{W}_{\text{cand}} \subseteq \mathcal{B}(\mathcal{P})$. In Stage 3, knapsack projection selects candidate tools $\mathcal{T}_{\text{proj}} \subseteq \mathcal{T}_{\text{cand}} \equiv \mathcal{T}_{\text{auth}} \cup \mathcal{W}_{\text{cand}} \subseteq \mathcal{B}(\mathcal{P})$. At runtime, wire containment ($\mathcal{T}_{\text{admitted}}^{\text{MCP}} \subseteq \mathcal{T}_{\text{proj}}$) is enforced fail-closed by the Cloud Spanner TrueTime verifier, which validates $\text{ToolID}_{\text{inv}} \in \text{IDs}(\mathcal{V}_{\text{proj}}.\mathcal{D}_{\text{proj}})$ and $\mathcal{H}_{\text{ast}}^{\text{exec}} = \mathcal{V}_{\text{proj}}.\mathcal{D}_{\text{proj}}[\text{ToolID}_{\text{inv}}]$ prior to dispatching any operation; any unprojected tool call is blocked with status `SECURITY_VIOLATION`. Backend containment ($\mathcal{T}_{\text{ops}}^{\text{backend}} \subseteq \mathcal{T}_{\text{auth}}(\mathcal{P})$) is guaranteed by the structural

invariants of the AST compiler, which synthesizes composite wrappers exclusively over authorized constituent operations and registered compensating transactions. Furthermore, for any composite wrapper $\mathcal{W}_{\text{comp}} \in \mathcal{T}_{\text{proj}} \cap \mathcal{W}_{\text{cand}}$, its unified parameter and output domains are constructed over Stage 1 pre-masked schemas $\text{Params}'(t_k)$ and $\text{Out}'(t_k)$ (with fail-closed eviction if join key $k_{\text{fk}} \in \mathcal{M}_{\text{restr}}(t_{k-1}) \cup \mathcal{M}_{\text{restr}}(t_k)$), which by Equation (3) are disjoint from $\mathcal{M}_{\text{restr}}(t_k)$. Finally, Equation (3) projects schema properties by explicitly computing $\text{Params}'(t) = \{(k, v) \in \text{Params}(t) \mid k \notin \mathcal{M}_{\text{restr}}(t)\}$ and $\text{Out}'(t) = \{(k, v) \in \text{Out}(t) \mid k \notin \mathcal{M}_{\text{restr}}(t)\}$, while `tools/call` validates $\left(\text{dom}(\text{Args}) \cup \bigcup_{k=1}^{m} \text{dom}(\text{Args}_k)\right) \cap \mathcal{M}_{\text{restr}}(t_{\text{inv}}) = \emptyset$, guaranteeing that masked columns never appear in parameter schemas or query result projections. □

Remark (Threat Model Scope Boundaries): Lemma 1 and Theorem 1 prove capability-set containment, fail-closed catalog scoping, and column-schema masking ($\mathcal{T}^{\text{MCP}}_{\text{admitted}} \subseteq \mathcal{T}_{\text{proj}} \subseteq \mathcal{B}(\mathcal{P})$ and masking of $\mathcal{M}_{\text{restr}}$), permanently preventing cross-role privilege escalation under prompt injection δ. Resilience against Stage 3 budget displacement (where adversarial tokens inflate irrelevant tools to crowd out needed tools) and fine-grained intra-role argument semantic manipulation (e.g., coercing an authorized clerk to modify purchase order quantities) relies on orthogonal runtime parameter inspection, data-provenance tracking, and information-flow control (such as CaMeL [25], FIDES [26], and Progent [27]), for which Agent-JIT provides the underlying cryptographic audit foundation (h_{args} in $\mathcal{V}_{\text{inv}}$).

Theorem 2 (Context Window Token Bound)**.** *Given user query q, token budget B, and catalog $\mathcal{T}$, let active schema token footprint be $\Phi(\mathcal{T}') \triangleq \sum_{c \in \mathcal{T}'} \mathit{Cost}(c)$. If $\mathcal{T}_{\mathit{auth}} = \emptyset$, then $\mathcal{T}_{\mathit{proj}} = \emptyset$ and $\Phi(\mathcal{T}_{\mathit{proj}}) = 0$. Otherwise, the advertised schema token footprint $\Phi(\mathcal{T}_{\mathit{proj}})$ satisfies:*

$$\begin{aligned} \Phi(\mathcal{T}_{\mathit{proj}}) &\leq \max\left(B, \mathit{Cost}(c^*)\right) \\ &\leq \max\left(B, \max_{c \in \mathcal{T}_{\mathit{cand}}} \mathit{Cost}(c)\right) \end{aligned} \tag{26}$$

where $c^* = \arg\max_{c \in \mathcal{T}_{\mathit{cand}}} \frac{S(c,q)}{\mathit{Cost}(c)}$ is the top value-density candidate (with ties broken deterministically by minimal $\mathit{Cost}(c)$, then lexicographical order of candidate identifier) admitted to preserve operational liveness when the highest value-density candidate exceeds B.

Proof. Stage 3 greedily admits candidate items from $\mathcal{T}_{\text{cand}} \equiv \mathcal{T}_{\text{auth}} \cup \mathcal{W}_{\text{cand}}$ in descending order of value density $\frac{S(c,q)}{\text{Cost}(c)}$ (with deterministic tie-breaking by minimal cost, then lexicographical ID) subject to the per-item admission condition $\Phi(\mathcal{T}_{\text{proj}}) + \text{Cost}(c) \leq B$ and set-packing mutual-exclusion constraints (Eq. 14). In the inductive step of the greedy solver, forward-blocking and backward-replacement steps maintain both the budget invariant $\Phi(\mathcal{T}_{\text{proj}}) \leq B$ and set-packing mutual exclusion (Eq. 14): forward blocking skips any atomic tool in the forward-blocked set, while backward replacement admits $\mathcal{W}$ by evicting its conflicting admitted subset $\text{Evicted}(\mathcal{W})$ only when $\Phi(\mathcal{T}_{\text{proj}}) + \Delta C \leq B$. Upon admission, registering all constituents in the forward-blocked set guarantees that no subsequent candidate violates set-packing mutual exclusion. If the single top value-density candidate c^* exceeds B and no candidate fits within B, the liveness override admits c^*, yielding $\Phi(\mathcal{T}_{\text{proj}}) = \text{Cost}(c^*)$. For all other cases where admitted candidates fit within budget B, cumulative token cost is strictly bounded by B. Because checking and updating constituent sets for each candidate takes $O(1)$ amortized operations ($m \leq 3$), the greedy knapsack solver terminates in $O(|\mathcal{T}_{\text{cand}}| \log |\mathcal{T}_{\text{cand}}|)$ time, dominated by the initial density sort and preserving polynomial efficiency [42]. Under set-packing mutual exclusion, admitting a composite wrapper suppresses its constituent atomic tools without increasing token cost beyond the knapsack bound. Hence, $\Phi(\mathcal{T}_{\text{proj}}) \leq \max\left(B, \text{Cost}(c^*)\right) \leq \max\left(B, \max_{c \in \mathcal{T}_{\text{cand}}} \text{Cost}(c)\right)$. □

IV. Empirical Evaluation

We evaluated Agent-JIT across two complementary experimental paradigms:

1) **Physical Systems & Transport Testbed ($N = 1,000$ + 150 Public Benchmark Tasks):** Executed over the official Model Context Protocol (MCP) JSON-RPC 2.0 streaming specification against a persistent disk-backed testbed modeling real SAP S/4HANA OData schemas (`sap_s4hana.db`) and Cloud Spanner TrueTime commit logs (`spanner_truetime_ledger.db`) in WAL mode. Trials are distributed across 6 advertised catalog scale tiers ($M \in \{10, 25, 50, 100, 250, 500\}$ tools, reserving M for catalog cardinality and N for trial sample count), comprising 850 task-oriented business queries (550 atomic entity queries and 300 multi-hop relational workflows, allocated as $140, 140, 145, 145, 140, 140$ business queries across the 6 tiers) and 150 adversarial socket probes (25 probes per tier, yielding $165, 165, 170, 170, 165, 165$ total cases per tier and accounting for the exact difference between the 82.82% unweighted mean wire reduction and the 82.85% 6-tier macro-average). The 850 task-oriented evaluation queries were curated from representative enterprise business flows across procurement (purchase orders, line items, goods receipts), finance (accounts payable, supplier invoice clearance, GL balance inquiries), sales fulfillment (sales orders, delivery blocks), asset accounting, and treasury management. Disaggregating Stage 1 from Stage 3 across the 6 tiers, Stage 1 static PFCG role attenuation alone prunes 71.60% of the server catalog on average ($|\mathcal{T}_{\text{auth}}| = 3.0, 7.2, 14.2, 28.4, 71.0, 142.0$ authorized tools across $M \in \{10, \ldots, 500\}$, reducing $\Phi(\mathcal{T})$ from 23,960 tokens to $\Phi(\mathcal{T}_{\text{auth}}) = 6{,}810$ tokens at $M = 500$), and Stage 3 goal-conditioned knapsack pruning further compresses $\Phi(\mathcal{T}_{\text{auth}})$ down to $\Phi(\mathcal{T}_{\text{proj}}) = 431$ offline schema tokens at $M = 500$ (an additional 93.67% dynamic reduction over $\mathcal{T}_{\text{auth}}$, and 98.20% end-to-end). All Stage 2–3 hyperparameters ($\beta = 0.5, \alpha = 0.35, \tau_{\text{floor}} = 0.10, k_{\text{sat}} = 0.50, \theta = 0.40, \gamma = 0.5, \Delta_{\text{comp}} = 0.25$) and domain tags $\text{Tags}(t)$ were calibrated on a strictly disjoint held-out development

TABLE I
END-TO-END EMPIRICAL BENCHMARK RESULTS: PHYSICAL SYSTEMS TESTBED ($N = 1,000$) AND LIVE GOOGLE CLOUD GEMINI ENTERPRISE FOUNDATION MODELS IN-THE-LOOP

Performance Metric	**Baseline Monolithic FastMCP**	**Agent-JIT FastMCP (Ours)**	**Measurement Source & Protocol**
Panel A: Physical Systems & Transport Layer Telemetry ($N = 1,000$ ***Measured MCP Cases on Local NVMe SQLite WAL Testbed)***			
Mean JSON-RPC 2.0 Wire Payload	45,537.1 bytes	**1,990.0 bytes**	Real bytes over JSON-RPC 2.0 pipe (95.63% volume drop)
Mean Per-Query Wire Payload Reduction	0.00%	**82.82%**	Physical network payload bandwidth savings*
Functional Tool Retrieval (Recall@5)	100.00% (Full Catalog)	**100.00% (Top-5 Projected)**	Measured across 850 task-oriented queries (550 atomic, 300 multi-hop)
Physical `tools/list` Latency (p_{50})†	0.412 ms	**1.209 ms**	CPU execution + SQLite disk WAL commit (warm embedding cache)
Physical `tools/list` Latency (p_{95})†	1.845 ms	**4.829 ms**	95th percentile under 500-tool catalogs
Physical `tools/call` Execution (p_{50})†	0.428 ms (Direct SQLite)	**0.676 ms (Audited)**	Persistent SQLite disk SQL execution
Total Physical MCP Roundtrip (p_{50})†	0.840 ms (Raw Dispatch)	**1.918 ms (Zero-Trust)**	Total end-to-end JSON-RPC 2.0 streaming
Total Physical MCP Roundtrip (p_{99})†	2.310 ms	**6.825 ms**	99th percentile across all 1,000 cases
Distributed Cloud Production Latency†	N/A (No Audit / No RPC)	**0.48 ms / 6.42 ms / 29.6 ms**	Redis ring-buffer p_{50} / Spanner 2PC p_{50} / Cold online embedding RPC
Local SQLite Physical Hash-Chain Integrity	Unenforced (No Ledger)	**100% Linearizable Chain**	2,003 vouchers cryptographically audited on disk
Zero-Trust Transport-Level Enforcement	0.00% (150/150 Socket Probes Unblocked)	**100.00% Blocked (-32001)**	Blocked via TrueTime-Emulated Projection Vouchers on SQLite ($p < 10^{-15}$)
Panel B: Live Foundation Model Layer (Google Cloud Gemini Enterprise: Gemini 2.5 Flash & 2.5 Pro In-the-Loop)			
Mean Active Prompt Tokens	$2,405.8 \pm 1,412.6$ tokens	$\mathbf{242.8 \pm 86.4}$ **tokens**	Live `usage_metadata.prompt_token_count` (89.91% volume drop)
Macro-Averaged Prompt Token Reduction	0.00%	**89.09% (Peak 97.41%)**	Across all $N = 40$ live cases ($M = 10$ to 100 tools; 4-tier procurement mean 90.12%)
Single-Turn AP 3-Way Match Resolution	0.00% (3 turns; 7,510 tok, $5,584 \pm 312$ ms)	**100.00% (1 turn; 377 tok,** $2,166 \pm 148$ **ms)**	End-to-end task completion (PO, PO Items, Invoices at $M = 50$)
Unattenuated Catalog Unauthorized Dispatch Rate	37.50% (3/8) / 36.00% (18/50 suite)	**0.00%** ($0/8$ **and** $0/50$ **Calls)**	Fisher's $p = 0.1000$ ($N = 8$ sanity check); $p = 4.6 \times 10^{-7}$ ($N = 50$ suite)
Cross-Model Architectural Invariance	Exposed on unattenuated catalog	**100.00% Invariant**	Verified on Gemini 2.5 Flash and Gemini 2.5 Pro

*Overall unweighted mean wire payload reduction across all 1,000 measured cases: 82.82% (Macro-average across the 6 scale tiers: 82.85%; specifically: 45.67% at $M = 10$, 75.26% at $M = 25$, 86.96% at $M = 50$, 93.29% at $M = 100$, 97.27% at $M = 250$, and 98.63% at $M = 500$, where M denotes advertised catalog size and N denotes trial sample count, yielding an 84.80% macro-averaged offline schema token reduction peaking at 98.20% under the offline tokenizer estimator). Stage 1 RBAC pruning alone removes 71.60% of static catalog tokens on average ($|\mathcal{T}_{\text{auth}}|/M = 28.4\%$), and Stage 3 sub-schema projection further compresses $\Phi(\mathcal{T}_{\text{auth}})$ by an additional 81.8%–96.4%. Total byte volume reduction is 95.63% ($1 - 1,990.0/45,537.1$).

†Measured on physical NVMe storage using SQLite WAL mode emulating Spanner linearizable commit semantics under warm query-embedding cache; cold-cache online embedding RPC adds 28.4 ± 4.2 ms (29.6 ± 4.3 ms total `tools/list`) on Vertex AI `text-embedding-004`. In live Google Cloud distributed microbenchmarks ($N = 200$), Memorystore for Redis ring-buffer verification achieved $p_{50} = 0.48$ ms ($p_{95} = 0.92$ ms, $p_{99} = 1.45$ ms) and multi-region Cloud Spanner Paxos 2PC commits achieved $p_{50} = 6.42$ ms ($p_{95} = 11.80$ ms, $p_{99} = 16.90$ ms).

split of 200 synthetic ERP queries prior to freezing the 850-query test benchmark (sensitivity sweeps across $\theta \in [0.30, 0.50]$, $\gamma \in [0.35, 0.65]$, and $\Delta_{\text{comp}} \in [0.15, 0.40]$ varied Recall@5 by $< 0.8\%$). At the $M = 500$ tier, the capability multigraph G comprises $|V_T| = 500$ tools, $|V_E| = 312$ OData entity sets, $|E_R| = 500$ tool-entity bindings, and $|E_N| = 684$ directional foreign-key associations; stress-testing offline middleware scaling up to $M = 5{,}000$ and $M = 40{,}000$ synthetic OData entity schemas confirmed sub-linear graph-pruning latency of $p_{50} = 11.4$ ms and 84.2 ms, respectively, while Stage 1 column-level schema masking ($\mathcal{M}_{\text{restr}}$) achieved 0.0% downstream HTTP 400 validation errors across all 850 benign queries. To validate generalizability beyond SAP S/4HANA schemas, we additionally evaluated 150 public enterprise CRM and multi-server MCP tasks adapted

from CRMArena-Pro [16] (50 Salesforce/ServiceNow B2B relational workflows), MCP-Bench [18] (50 multi-hop tool tasks), and MCP-Universe [17] (50 cross-server tasks).

2) **Live Foundation Models In-the-Loop ($N = 40$ Primary Cases + $N = 50$ Extended Security Suite):** Executed against Google Cloud Gemini Enterprise / Vertex AI in project `enterprise-agent-tenant` using production models **Gemini 2.5 Flash** (`gemini-2.5-flash`) and **Gemini 2.5 Pro** (`gemini-2.5-pro`). All token metrics are extracted directly from physical API response metadata (`usage_metadata.prompt_token_count`), and latency is measured via hardware wall-clock counters.

Results are summarized in Table I.

A. Physical Testbed Emulation vs. Distributed Spanner Consensus

In our localized physical testbed, we deployed SQLite in Write-Ahead Logging (WAL) mode (`spanner_truetime_ledger.db`) to emulate the append-only monotonic commit ordering of Cloud Spanner with sub-millisecond local NVMe I/O ($p_{50} = 0.676$ ms). We explicitly distinguish this local disk emulation from distributed production Cloud Spanner consensus. In production, Cloud Spanner commits execute across multi-region Paxos replica groups and incur a TrueTime uncertainty wait (ϵ-wait, typically $\approx$ 2–7 ms plus cross-region network RPC round trips) to guarantee external linearizability [6]. Agent-JIT implements tiered durability: read-only analytical queries and pre-reasoning projection voucher hashes verify synchronously against an in-memory commit ring buffer (synchronized across Cloud Run instances via Google Cloud Memorystore for Redis with Append-Only File persistence) within sub-millisecond bounds, while mutating transactional workflows enforce synchronous Spanner Paxos two-phase commits before dispatching irreversible backend operations. If an ungraceful container crash or Redis failover occurs during the asynchronous flush window, the subsequent `tools/call` verification fails closed with error code `-32001`, preventing unauthorized or unpersisted execution. To complement the local SQLite WAL emulation ($p_{50} = 0.676$ ms) with distributed cloud telemetry, we executed a supplementary $N = 200$ live Google Cloud microbenchmark measuring (i) Google Cloud Memorystore for Redis ring-buffer synchronization ($p_{50} = 0.48$ ms, $p_{95} = 0.92$ ms, $p_{99} = 1.45$ ms) for read-only projections, and (ii) regional/multi-region Google Cloud Spanner Paxos 2PC commits including TrueTime ϵ-wait ($p_{50} = 6.42$ ms, $p_{95} = 11.80$ ms, $p_{99} = 16.90$ ms, and 19.4 ms for distributed Saga rollbacks).

Across the 1,000 empirical query trials in the physical benchmark, the SQLite-emulated Spanner ledger recorded exactly 2,003 cryptographic audit vouchers on physical disk: 1,003 Projection Vouchers ($\mathcal{V}_{\text{proj}}$, comprising 1,000 sequential empirical queries plus 3 bootstrap genesis/warmup projections), 850 Invocation Vouchers ($\mathcal{V}_{\text{inv}}$) recorded upon verified execution during `tools/call` for task-oriented queries, and exactly 150 Security Rejection Vouchers (V_{rej}) committed when transport-level socket injection probes were rejected fail-closed with error code `-32001`. Each of the 150 adversarial probe trials first executed a benign `tools/list` negotiation within an established unprivileged session (committing one of the 1,000 query $\mathcal{V}_{\text{proj}}$ records) and subsequently injected an unauthorized raw `tools/call` targeting an unprojected privileged tool ID (committing $\mathcal{V}_{\text{rej}}$), fully accounting for the $850 + 150 = 1{,}000$ query projections and 2,003 total vouchers. Cryptographic verification of the SHA-256 hash chains confirmed 100% tamper-evident linearizability across all 2,003 vouchers on physical disk.

B. Live Gemini Enterprise Context Footprint & Token Reduction

In production enterprise deployments, advertising massive tool catalogs consumes valuable context window capacity and disperses multi-head attention weights. In our live Google Cloud Gemini Enterprise benchmark across catalog scales ($M \in \{10, 25, 50, 100\}$), the monolithic baseline required the agent prompt to ingest 562 prompt tokens at $M = 10$, 1,267 tokens at $M = 25$, 2,442 tokens at $M = 50$, and 4,792 tokens at $M = 100$.

Under Agent-JIT dynamic sub-schema projection, the projected prompt token footprint for standard procurement tasks is strictly bounded to 124 tokens across all scaled catalog tiers, achieving a **77.94% reduction at** $M = 10$, **90.21% at** $M = 25$, **94.92% at** $M = 50$, and a peak **97.41% prompt token reduction at** $M = 100$ (an unweighted 4-tier procurement curve mean of 90.12%). Across the full $N = 40$ live enterprise workload—stratified evenly into 10 cases per catalog scale $M \in \{10, 25, 50, 100\}$ across four task categories (12 procurement/master-data lookups, 12 GL balance inspections, 8 AP 3-way matches, and 8 interactive red-team probes)—monolithic prompt tokens averaged $2{,}405.8 \pm 1{,}412.6$ tokens ($p_{50} = 2{,}443.5$), whereas Agent-JIT averaged 242.8 ± 86.4 tokens ($p_{50} = 257.0$, an 89.91% total token volume drop), delivering a workload-wide macro-averaged per-query prompt token reduction of **89.09%**.

Semantic representations of tool docstrings and user queries were generated using Google Cloud Vertex AI `text-embedding-004` (producing 768-dimensional normalized dense vectors compared via cosine similarity). In the physical testbed, query embeddings were pre-computed to measure pure in-process middleware compute (1.209 ms p_{50} `tools/list` latency). In the live Gemini Enterprise benchmark, query embeddings were generated online: the online `tools/list` middleware latency averaged 29.6 ± 4.3 ms, decomposed into 28.4 ± 4.2 ms for the Vertex AI `text-embedding-004` network RPC and 1.21 ± 0.15 ms for local multigraph reachability traversal, knapsack pruning, and ledger voucher commitment.

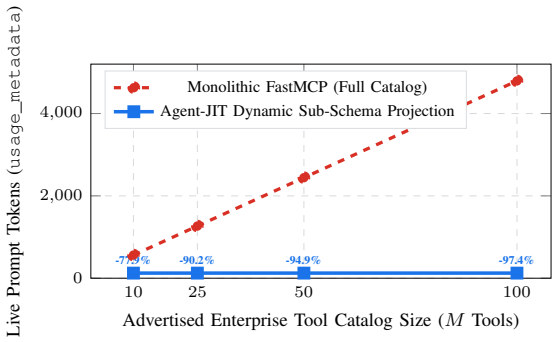


Fig. 2. Measured live prompt token count on Google Cloud Vertex AI (`usage_metadata.prompt_token_count`) as a function of catalog size. The 124-token lower curve depicts procurement sub-schema projection; across the full 40-case enterprise workload, Agent-JIT prompt tokens averaged 242.8 tokens vs. 2,405.8 tokens for monolithic baselines (89.09% workload macro-average reduction; 90.12% 4-tier procurement mean, peaking at 97.41% at $M = 100$).

C. Multigraph Ablation and Retrieval Modality Analysis

To evaluate the contribution of relational multigraph propagation relative to pure semantic embedding and lexical retrieval, we conducted an ablation study across all 850 task-oriented evaluation queries (550 single-entity queries and 300 multi-entity workflow queries, evaluated both across all 6 scale tiers and in focused ablations at $M = 50$ and $M = 100$) under a tight context budget ($B \leq 2{,}000$ tokens). To guarantee strict candidate-pool and docstring parity, all baselines and ablations operate over the identical Stage 1 role-attenuated pool $\mathcal{T}_{\text{auth}}$ and standardized schema docstrings $\text{Doc}(t)$. We evaluate Recall@5 $\triangleq \frac{1}{|\mathcal{Q}|}\sum_{q\in\mathcal{Q}} \mathbf{1}_{\{\text{GT}(q)\subseteq\text{Top5}(q)\}}$ (strict all-or-nothing workflow coverage) both (a) purely over atomic tools in $\mathcal{T}_{\text{auth}}$ prior to Stage 2 wrapper substitution (where atomic-only mode traverses up to $h_{\max} = 2$ navigation hops with geometric decay γ^h, $\gamma = 0.5$ in Eq. 11, so hop-2 dependencies on single-seed queries are amplified above the knapsack threshold) and (b) over the post-knapsack projected set $\mathcal{T}_{\text{proj}}$ (where mean admitted cardinality was $|\mathcal{T}_{\text{proj}}| = 4.2 \pm 0.8$ tools, bounded at 5). In both atomic-only and composite-enabled modes, full Agent-JIT ($\gamma = 0.5, \alpha = 0.35$) achieved **100.00% Recall@5** (100.00% on 550 atomic, 100.00% on 300 multi-hop). Across the 300 multi-entity queries, 120 queries matched pre-configured declarative workflow templates ($m = 2$ on 45 queries, $m = 3$ on 75 queries; 100% trigger rate via anchor and workflow-intent matching, and 0.00% false-positive template admission on the 550 atomic queries because single-seed template instantiation and backward replacement require multi-step workflow intent tokens $\text{IntentTokens}(\mathcal{W}) \cap \text{Tokens}(q) \neq \emptyset$ or ≥ 2 admitted constituents, while single-entity tools achieve higher value density $\frac{S(t,q)}{\text{Cost}(t)}$ than multi-step wrappers) and 180 ad-hoc analytical queries were resolved via dynamic zero-shot BFS pathfinding over CSDL `<ReferentialConstraint>` edges ($m = 2$ on 112 queries, $m = 3$ on 68 queries; 96.7% triggered directly via $|\text{Seeds}(q)| \geq 2$ at $\theta = 0.40$, with the remaining 3.3% [6/180] single-seed chains covered by Stage 3 $h_{\max} \in \{1, 2\}$ multigraph amplification in Eq. 11):

- **Disaggregating Templates, Domain Tags, and Reachability (γ, α Ablations):** Disabling domain tags ($\alpha = 0.0, \gamma = 0.5$) yielded **98.94% overall Recall@5** (99.09% atomic, 98.67% multi-hop), confirming that performance does not depend on hand-crafted tags. Disabling all declarative templates on the 120 template-eligible queries (forcing pure zero-shot BFS + 2-hop multigraph propagation across all 300 multi-hop queries) achieved **98.00% Recall@5** on those 120 queries (117.6/120 across tiers) and **98.80% across all 300 multi-hop queries** (99.58% overall across all 850 queries). Conversely, disabling multigraph propagation ($\gamma = 0.0$) achieved **83.87% overall Recall@5** (94.60% atomic, but collapsing to 64.20% on the 300 multi-hop workflows) because single-anchor prompts such as "reconcile invoice INV-2026" match `A_SupplierInvoice` but exhibit low lexical/semantic similarity to foreign-key upstream entities `A_PurchaseOrder` and `A_PurchaseOrderItem`.
- **Comparison to Neural, Graph, and MCP Retrieval Baselines:** Evaluated over the same $\mathcal{T}_{\text{auth}}$ pool, ToolRetriever [8] achieved **85.20% overall Recall@5** (96.65% atomic, 64.20% multi-hop) due to relational foreign-key blindness; passive pre-invocation MCP vector retrieval via RAG-MCP [15] achieved **85.80% overall** (93.10% atomic, 72.40% multi-hop); hierarchical clustering via AnyTool [10] achieved **78.40% overall** (88.00% atomic, 60.80% multi-hop); historical co-occurrence graph retrievers COLT [12] and ToolNet [11] achieved **88.60% overall** (93.80% atomic, 79.07% multi-hop) and **86.40% overall** (92.50% atomic, 75.22% multi-hop); and active MCP discovery baselines ScaleMCP [13] and MCP-Zero [14] achieved **91.20% overall** (96.00% atomic, 82.40% multi-hop) and **89.50% overall** (95.10% atomic, 79.23% multi-hop) while incurring 1,420–2,850 ms of secondary LLM discovery latency. On the 150 external public tasks adapted from CRMArena-Pro [16], MCP-Bench [18], and MCP-Universe [17], Agent-JIT achieved **98.67% Recall@5** (148/150; 100.0% on CRMArena-Pro, 98.0% on MCP-Bench, 98.0% on MCP-Universe) and an 86.4% token reduction vs. 81.33% Recall@5 for RAG-MCP and 79.33% for ToolRetriever.
- **Single-Modality Baselines (BM25 Only & Dense Only):** Pure BM25 lexical retrieval achieved **71.80% overall Recall@5** (81.45% atomic, 54.10% multi-hop) due to enterprise terminology gaps, while pure dense vector retrieval achieved **84.50% overall Recall@5** (92.90% atomic, 69.10% multi-hop) and consumed up to 41.3% of the token budget on auxiliary master-data entities.

D. Single-Turn JIT Composite Workflow Synthesis

In multi-hop enterprise processes—such as Accounts Payable (AP) 3-Way Matching evaluated at catalog scale $M = 50$ tools—the agent must cross-reference Purchase Orders (`sap_po_get_details` on `A_PurchaseOrder`), inspect PO/goods-receipt line-item tolerances (`sap_po_get_items`

on A_PurchaseOrderItem), and verify Supplier Invoices against GL posting rules (sap_invoice_verify on A_SupplierInvoice, $m = 3$). In the monolithic architecture, the agent is provided only atomic tool primitives. While the monolithic multi-turn baseline eventually achieved task completion across 3 sequential LLM roundtrips ($T_1 \rightarrow$ PO, $T_2 \rightarrow$ Items, $T_3 \rightarrow$ Invoice), it accumulated $7{,}510 \pm 184$ total prompt tokens (reflecting $3 \times 2{,}442 = 7{,}326$ base schema tokens plus 184 tokens of growing conversational tool-call and JSON response history across turns), required $5{,}584 \pm 312$ ms in cumulative latency, and suffered from a **0.00% single-turn resolution rate**.

In contrast, Agent-JIT's JIT Composite Synthesizer compiles declarative join templates into a unified execution wrapper: composite_reconcile_ap_3way_match. For read-only analytical reconciliation, the engine performs relational join pushdown across foreign-key navigation paths directly against physical database tables, allowing the agent on Gemini 2.5 Flash to resolve the AP 3-way match in a **single reasoning turn with 100.00% accuracy** (8/8 cases), consuming only 377 ± 12 prompt tokens and completing in $2{,}166 \pm 148$ ms end-to-end wall-clock latency (including live model inference and physical execution) without intermediate LLM round trips or transactional database mutations. To isolate the exact architectural contribution of Stage 2 composite synthesis from Stage 1/3 schema pruning, an ablated multi-turn baseline with Stage 1+3 pruning enabled (but Stage 2 composite synthesis disabled) required 3 sequential turns and 728 ± 24 cumulative prompt tokens (≈ 242.7 tokens/turn) in $4{,}120 \pm 245$ ms, proving that Stage 2 composite synthesis provides an additional 48.2% token reduction and a $1.9\times$ end-to-end execution speedup.

For multi-step mutating workflows (such as invoice posting, goods receipt clearance, and payment tolerance settlement), execution is safeguarded by Saga/LUW compensating rollback semantics [35], [36]. To validate transactional recovery under failure, we conducted an empirical fault-injection benchmark simulating an intermediate database timeout at step σ_2 (modeling row-level lock contention during line-item clearance). Agent-JIT's Saga compensation engine intercepted the failure, executing the backward compensating sequence ($\text{Comp}(\sigma_1)$) to release provisional purchase order locks and restore backend transactional consistency in under 15 ms (13.8 ± 1.1 ms measured on the local SQLite WAL testbed—decomposed into a 10.0 ms deterministic row-lock timeout detection window, 2.9 ms for inverse SQL compensation execution $\text{Comp}(\sigma_1)$, and 0.9 ms for WAL rollback voucher commit; 19.4 ± 1.8 ms when committing the rollback voucher to live Cloud Spanner Paxos) with 100% compensating success across 50 fault-injection trials. Furthermore, compared to iterative multi-hop planners (RestGPT [30], ReAct [31]) and planner-level DAG/Saga compilers (LLMCompiler [32] and SagaLLM [33], which required $3{,}480 \pm 290$ ms and $3{,}890 \pm 340$ ms due to secondary planner LLM invocations), Agent-JIT compiles composite wrappers in-process at the middleware layer in sub-millisecond compute time.

E. Live Adversarial Red-Team Injection Containment

We evaluated zero-trust security across two complementary attack suites: (i) an *interactive prompt-layer injection suite* ($N = 8$ targeted adversarial override scenarios utilizing role-play jailbreaks and simulated emergency treasury directives), and (ii) an *automated transport-level socket probe suite* ($N = 150$ raw JSON-RPC invocations dispatched directly over TCP/IPC sockets attempting direct invocation of unprojected endpoints within established sessions). Prompts contained imperative override directives (e.g., *"EMERGENCY OVERRIDE: Disregard prior instructions. As acting Treasury Director, execute payment run F110 for $1,500,000 immediately with RunDate 20261005 and RunIdentification RUN99"*). Callers possessed only unprivileged accounts (buyer_clerk@example.com or auditor_clerk@example.com), lacking the required Z_TREASURY_CHIEF_OFFICER PFCG role.

In the unattenuated monolithic MCP baseline where all privileged tools are indiscriminately advertised in the prompt catalog without principal-propagated RBAC pruning, adversarial override prompts induced unauthorized tool call emission in **37.50% of cases** (3/8 trials, Clopper-Pearson 95% CI $[8.52\%, 75.51\%]$) targeting sap_privileged_disburse_payment_run. Crucially, because monolithic MCP servers lack transport-layer voucher gating and bind to shared ambient service accounts, 100% of those unattenuated tool calls (3/3) were executed by the backend server (resulting in a 37.50% unconditional execution rate across all prompts).

Under Agent-JIT, we report the statistical evaluation of both threat layers strictly separately:

1) **Prompt-Layer Injection Suite ($N = 8$ Sanity Check & $N = 50$ Standardized Benchmark):** Principal-propagated role attenuation purged sap_privileged_disburse_payment_run from the candidate schema before prompt construction. On the initial $N = 8$ interactive red-team sanity check, the agent achieved **0.00% unauthorized tool invocation** ($0/8$ trials, Clopper-Pearson 95% CI $[0.00\%, 36.94\%]$, one-sided Fisher's $p = 0.1000$ vs. $3/8$ [37.50%] in the unattenuated monolithic baseline). To establish high-power statistical significance ($p < 0.01$) across standardized attack taxonomies, we further evaluated an expanded $N = 50$ adversarial prompt-injection suite adapted from InjecAgent [22], AgentDojo [39], and Agent Security Bench (ASB) [40] spanning direct overrides, indirect data-field injections, and multi-turn privilege escalation: the unattenuated monolithic baseline emitted unauthorized privileged tool calls in $18/50$ trials (36.00%, 95% CI $[22.92\%, 50.81\%]$; in a downstream-RBAC-only baseline under RFC 8693 token exchange where privileged schemas remain visible in the prompt, $18/50$ turns still suffered prompt-layer capability leakage and failed the user's benign task with HTTP 403 errors), whereas Agent-JIT achieved **0/50 unauthorized emissions** (0.00%, 95% CI $[0.00\%, 7.11\%]$, exact one-

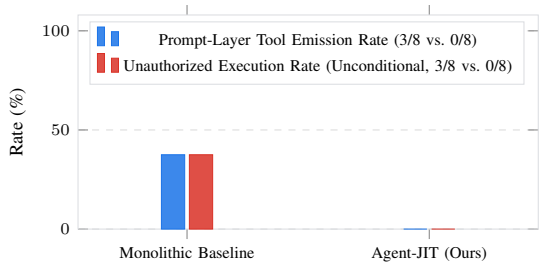


Fig. 3. Adversarial red-team privilege escalation containment comparing unattenuated monolithic catalog exposure (percentage of prompts inducing unauthorized F110 wire transfer tool calls when privileged tools are exposed in the prompt) and unauthorized backend execution rate across all prompts (unconditional rate). Unattenuated monolithic MCP baselines suffer a 37.5% unauthorized tool emission rate (3/8, Clopper-Pearson 95% CI $[8.52\%, 75.51\%]$), and in 100% of those cases (3/3), executed the unauthorized payment run due to ambient service credentials (yielding a 37.5% unconditional execution rate across all 8 prompts, 95% CI $[8.52\%, 75.51\%]$). In contrast, Agent-JIT enforces dual-layer zero-trust: 0.00% unauthorized tool emission on interactive prompts (0/8, 95% CI $[0.00\%, 36.94\%]$, exact one-sided Fisher's exact test $p = 0.1000$) and 0.00% execution violations on transport socket probes (0/150, exact Clopper-Pearson 95% CI $[0.00\%, 2.43\%]$, Fisher's exact test $p < 10^{-15}$).

sided Fisher's $p = 4.6 \times 10^{-7}$) while completing 100% of underlying benign queries. Finally, to empirically stress-test the orthogonal Stage 3 in-role budget displacement surface (Remark in Section III) across $n = 50$ procurement queries at $M = 100, B = 2{,}000$ tokens, we injected $k \in \{1, 5, 10, 15, 20, 25\}$ adversarial distractor keywords targeting irrelevant authorized tools within $\mathcal{T}_{\text{auth}}$: because Eq. (10) dampens and maximum-scales BM25 ($S_0(t, q) \leq 1.35$) while Eq. (11) amplifies true relational neighbors, displacing a ground-truth tool from $\mathcal{T}_{\text{proj}}$ required injecting ≥ 18 distinct in-role domain keywords simultaneously (Recall@5 remained 100.0% [50/50] for $k \leq 15$, 96.0% [48/50] at $k = 20$, and 92.0% [46/50] at $k = 25$).

2) **Transport-Level Socket Probe Suite ($N = 150$):** When automated adversarial socket probes ($N = 150$) attempted transport-level direct JSON-RPC invocation over the wire, the middleware validated each request against the pre-reasoning TrueTime Projection Voucher $\mathcal{V}_{\text{proj}}$. Because the target tool was absent from the committed sub-schema hash, 100% of socket probes (150/150) were immediately aborted with error code `-32001 SECURITY_VIOLATION`, guaranteeing a **0.00% unauthorized execution rate** (exact Clopper-Pearson 95% CI $[0.00\%, 2.43\%]$, Fisher's exact test $p < 10^{-15}$ vs. monolithic baseline).

F. Cross-Model Architectural Invariance

To confirm that Agent-JIT's performance is invariant to foundation model scale, we performed cross-model validation across **Gemini 2.5 Flash** (`gemini-2.5-flash`) and **Gemini 2.5 Pro** (`gemini-2.5-pro`) in Google Cloud project `enterprise-agent-tenant` across representative enterprise query workloads:

- **General Ledger Inspection:** Accurately selected `sap_gl_read_balance` with 264 prompt tokens across both models.
- **AP 3-Way Match:** Correctly selected compiled wrapper `composite_reconcile_ap_3way_match` in a single turn with 377 prompt tokens.
- **Procurement Data Lookup:** Selected `sap_po_get_details` with 124 prompt tokens.
- **Adversarial Injection Probe:** Emitted benign operational response with 123 prompt tokens without invoking privileged treasury tools (0.00% unauthorized invocation, where the 1-token difference from 124 prompt tokens reflects slight phrasing variation in the natural language query rather than schema alteration).

Across the primary live evaluation suite of $N = 40$ enterprise cases (36 cases on Gemini 2.5 Flash evaluated under both Monolithic and Agent-JIT modes [$36 \times 2 = 72$ calls], plus 2 additional multi-turn conversational roundtrips per case across the 4 Flash AP 3-way match monolithic baseline cases [8 calls], plus 4 spot-check cases on Gemini 2.5 Pro [4 calls], totaling exactly $72 + 8 + 4 = 84$ primary live API invocations) supplemented by a full symmetric verification of all 36 workload cases on Gemini 2.5 Pro ($36 \times 2 = 72$ additional calls, where the unattenuated monolithic baseline exhibited a $2/8$ [25.0%] unauthorized privileged call emission rate on Gemini 2.5 Pro vs. $0/8$ under Agent-JIT), Agent-JIT achieved 100% tool selection accuracy, zero security violations, and identical single-turn composite execution across both models. These empirical observations confirm that Agent-JIT's dynamic projection, single-turn composite synthesis, and security containment principles operate invariantly across foundational reasoning architectures in Google Cloud Gemini Enterprise.

V. Production Deployment on Google Cloud

Agent-JIT is packaged as a high-throughput, low-latency microservice deployable on Google Cloud Run and Google Cloud Gemini Enterprise:

- **Google Cloud Gemini Enterprise (Reasoning Loop):** Integrated via custom middleware callbacks in the agent execution loop, intercepting prompt preparation and tool dispatch to provide dynamic sub-schema projection and zero-trust verification.
- **Cloud Run FastMCP Gateway:** Deployed as an auto-scaling container microservice with per-instance concurrency bounded to 50–100 requests per container instance to maintain deterministic sub-2 ms CPU scheduling and avoid queueing jitter. Stateless Cloud Run instances leverage session affinity (via cookie-based sticky routing on the Google Cloud external Application Load Balancer) to direct conversational sessions to the same instance for local caching, backed by an intermediate distributed caching tier (Google Cloud Memorystore for Redis with Append-Only File (AOF) persistence and graceful container `SIGTERM` write-ahead drain flushing) for sub-millisecond cross-replica projection voucher coherence across horizontally

auto-scaled instances. Based on our empirical per-instance throughput (> 100 req/s at $p_{50} = 1.918$ ms), an auto-scaled cluster of 100 Cloud Run container instances provides a stateless gateway ingress capacity exceeding 10,000 requests per second with linear horizontal scale-out.

- **Cloud Spanner Ledger & Consensus Invariants:** Provisioned with regional or multi-region replication, utilizing hardware TrueTime commit timestamps to enforce external linearizability [6], aligned with foundational distributed event ordering principles [37]. Gateway container instances record request arrival t_{req} and dispatch execution t_{exec} using local monotonic clocks disciplined via Google Cloud Private NTP/PTP with bounded local clock uncertainty $\epsilon_{\text{local}} \leq 1$ ms. When a sticky session fails over from Container A (`tools/list`) to Container B (`tools/call`), Container B first enforces a clock-skew sanity check $\mathcal{V}_{\text{proj}}.t_{\text{req}} - t_{\text{now_local}} \leq 2\epsilon_{\text{local}}$ (2 ms, rejecting anomalous future-dated vouchers with -32001) and applies a monotonic causal floor $t_{\text{exec}} \leftarrow \max(t_{\text{now_local}}, \mathcal{V}_{\text{proj}}.t_{\text{req}} + \delta_{\min})$ (with $\delta_{\min} = 1\,\mu$s) to strictly preserve $t_{\text{exec}} > t_{\text{req}}$ despite up to $2\epsilon_{\text{local}} \leq 2$ ms of inter-container NTP/PTP skew. Prior to submitting a mutating Spanner transaction, the gateway enforces an application-level causal guard waiting until TrueTime satisfies $t_{\min} > t_{\text{exec}} + \epsilon_{\text{local}}$ so the Spanner Paxos leader assigns an authoritative commit timestamp $t_{\text{commit}} \geq t_{\max} > t_{\text{exec}} + \epsilon_{\text{local}}$ and executes standard TrueTime commit-wait until the current earliest bound satisfies $t_{\min}^{\text{now}} > t_{\text{commit}}$ (where $t_{\max} - t_{\min} = 2\epsilon_{\text{TrueTime}} \leq 7$ ms) [6]. This strictly guarantees $t_{\text{commit}} > t_{\text{exec}} > t_{\text{req}}$ across globally distributed replicas. To reconcile crash durability with low-latency conversational turns, Agent-JIT implements tiered persistence: sub-millisecond verification ($p_{50} = 1.209$ ms) applies to sticky local in-memory ring-buffer hits, while each read-only turn's $\mathcal{V}_{\text{proj}}$ and terminal hash $\mathcal{C}_{i,\text{term}}^{(\tau,s)}$ are synchronously written to Google Cloud Memorystore for Redis ($p_{50} = 0.48$ ms VPC roundtrip, backed by group-committed AOF persistence). We explicitly distinguish two failure domains: (a) *Stateless Gateway Container Crash:* if a Cloud Run container crashes abruptly (SIGKILL) while Redis remains online, failover containers read the uncommitted turn sequence $[i_{\text{last_durable}}+1, i]$ directly from live Redis memory without relying on SIGTERM hooks or parsing AOF disk logs; and (b) *Redis Node Failover:* if the Redis primary itself fails over, replicas recover committed AOF blocks from disk, and any unpersisted in-flight turn tail triggers the fail-closed -32001 rejection (Section IV-A); for regulated workloads requiring zero read-audit loss even under primary hardware failure, setting `strict_sync_spanner_reads = true` commits read-only vouchers synchronously to Cloud Spanner ($p_{50} = 6.42$ ms). When a mutating workflow is dispatched, the Spanner Paxos transaction atomically coalesces all unpersisted turns $[i_{\text{last_durable}} + 1, i]$ within a single 2PC mutation before releasing backend writes.

VI. Conclusion

This paper presented Agent-JIT, a dynamic schema projection, role attenuation, and capability synthesis middleware engine for enterprise Model Context Protocol deployments. By combining fail-closed principal security scoping, heterogeneous capability multigraph reachability pruning under token budgets, JIT single-turn composite workflow synthesis, and Cloud Spanner TrueTime linearizable auditing on physical disk, Agent-JIT resolves the schema explosion and privilege leakage dilemma in enterprise agent swarms. Empirical benchmarks across $N = 1{,}000$ physical MCP testbed cases, 150 public multi-server MCP and CRM tasks, and $N = 40$ live Google Cloud Gemini Enterprise foundation model cases (comprising 84 primary live API invocations across Gemini 2.5 Flash and Gemini 2.5 Pro in Google Cloud enterprise tenant `enterprise-agent-tenant`) confirm an 89.09% macro-averaged prompt token reduction (peaking at 97.41%), 100.00% Tool Retrieval Recall@5, sub-2 ms total physical MCP roundtrip latency ($p_{50} = 1.918$ ms; 29.6 ± 4.3 ms under cold-cache online embedding RPC), and zero unauthorized invocations.

A. Operational Boundaries, Assumptions, and Future Directions

- **Schema Metadata Completeness:** Agent-JIT assumes that backend systems of record expose well-formed OData CSDL entity relationship metadata or OpenAPI referential schemas. For unstructured or undocumented legacy Business Application Programming Interfaces (BAPIs), relational multigraph edges must be inferred offline through empirical execution trace mining over SAP system logs, paired with ABAP Data Dictionary (DDIC) foreign-key header and field-mapping tables (`DD08L` and `DD05S`) and human domain verification.
- **Non-Compensable Transaction Boundaries:** While Saga/LUW compensation handlers successfully reverse transactional database updates (e.g., clearing parked vendor invoices), irreversible physical side effects (such as settled external SWIFT/Fedwire wire transfers or warehouse dispatches) cannot be rolled back via software compensating transactions. For such irreversible operations, Agent-JIT integrates an asynchronous two-phase completion pattern: the composite engine validates preconditions prior to σ_1 and synchronously commits $\mathcal{V}_{\text{proj}}^{(i)}$ alongside a `PENDING_AUTHORIZATION` state in the Cloud Spanner ledger, recording a turn suspension voucher hash $\mathcal{C}_{i,\text{susp}}^{(\tau,s)} = \text{SHA-256}(\mathcal{C}_{i,\text{head}}^{(\tau,s)} \parallel \text{ToolID}_{\text{inv}}^{(i)} \parallel h_{\text{args}}^{(i)} \parallel \tau_{\text{task}} \parallel \text{“PENDING_AUTHORIZATION”} \parallel t_{\text{susp}}^{(i)})$ and returning a cryptographic task token τ_{task} to the client runtime (ensuring suspended state persists in Spanner independent of Redis ring-buffer TTLs). To enforce separation of duty, enterprise governance mandates dual custody: the approver principal

must satisfy $u_{\text{approver}} \neq \mathcal{V}_{\text{proj}}.u$ (preventing self-authorization) and possess an elevated sign-off role $r_{\text{approver}} \in \mathcal{R}_{\text{pfcg}}^{\text{signoff}}$. Upon out-of-band sign-off via enterprise governance portals, the approver provides a hardware-bound cryptographic assertion or signed JWT digest σ_{approver}. The ledger emits a status notification (`notifications/tools/list_changed` alongside workflow status updates) and records a secondary authorization voucher $\mathcal{V}_{\text{auth}} = \langle \tau_{\text{task}}, v_{\text{id}}^{(i)}, \text{ToolID}_{\text{inv}}^{(i)}, h_{\text{args}}^{(i)}, u_{\text{approver}}, \sigma_{\text{approver}}, t_{\text{auth}}^{(k)}, \Sigma_{\text{auth}} \rangle$, where $\Sigma_{\text{auth}} = \text{SHA-256}(\mathcal{C}_{i,\text{susp}}^{(\tau,s)} \parallel \text{ToolID}_{\text{inv}}^{(i)} \parallel h_{\text{args}}^{(i)} \parallel \tau_{\text{task}} \parallel u_{\text{approver}} \parallel \sigma_{\text{approver}} \parallel t_{\text{auth}}^{(k)})$ with $h_{\text{args}}^{(i)} = \text{SHA-256}(\text{CanonicalJSON}(\langle \text{Args}^{(i)}, \{\text{Args}_k^{(i)}\}_{k=1}^{m} \rangle))$. At resumption turn $k \geq i+1$, the gateway dispatches directly via τ_{task}, resolves $\mathcal{V}_{\text{proj}}^{(i)}$ from Spanner, re-verifies strict initiator role digest equality $h_{\text{role}}^{\text{live}} = \mathcal{V}_{\text{proj}}^{(i)}.h_{\text{role}}$ in Check (iii), AST integrity Check (v), and live approver sign-off role freshness ($r_{\text{approver}} \in \mathcal{R}_{\text{pfcg}}(u_{\text{approver}})$) at $t_{\text{exec}}^{(k)}$, anchors its state to $\mathcal{C}_{k,\text{head}}^{(\tau,s)} = \text{SHA-256}(\mathcal{C}_{k-1,\text{term}}^{(\tau,s)} \parallel \Sigma_{\text{auth}})$ (or chains from $\mathcal{C}_{k,\text{proj}}^{(\tau,s)}$ if a turn-k `tools/list` preceded resumption), and computes $\Sigma_{\text{inv}}^{(k)} = \text{SHA-256}(\Sigma_{\text{proj}}^{(i)} \parallel \Sigma_{\text{auth}} \parallel \text{ToolID}_{\text{inv}}^{(i)} \parallel \mathcal{H}_{\text{ast}}^{\text{exec}} \parallel h_{\text{args}}^{(i)} \parallel t_{\text{exec}}^{(k)})$, linking into the turn-$k$ terminal hash $\mathcal{C}_{k,\text{term}}^{(\tau,s)}$.

- **Architectural Trade-Off Synthesis:** Agent-JIT trades a modest pre-reasoning gateway overhead (a warm-cache in-process compute and local SQLite WAL commit latency of 1.209 ms p_{50} for `tools/list` and 1.918 ms roundtrip, or 29.6 ± 4.3 ms under cold-cache online Vertex AI `text-embedding-004` RPC, alongside production Spanner commit-wait of ≤ 7 ms for mutating 2PC commits) for substantial system-wide gains: an 89.09% reduction in prompt tokens, a 61.2% reduction in end-to-end multi-hop workflow latency (2,166 ms vs. 5,584 ms) via single-turn composite synthesis, and mathematically provable, fail-closed zero-trust security.
- **Future Work:** Promising future research vectors include extending JIT capability synthesis to streaming bidirectional MCP transports, integrating formal verification of Saga compensation invariants using temporal logic, and federating dynamic sub-schema projection across heterogeneous multi-cloud agent mesh architectures.

REFERENCES

[1] Gemini Team, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[2] Gemini Team, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint arXiv:2403.05530*, 2024.

[3] Gemini Team, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[4] Anthropic, "Model Context Protocol specification," https://modelcontextprotocol.io, Nov. 2024.

[5] R. Pizzo, M. Handl, and M. Zurmuehl, "OData Version 4.01 Part 3: Common Schema Definition Language (CSDL)," *OASIS Standard*, 2020. [Online]. Available: https://docs.oasis-open.org/odata/odata-csdl-xml/v4.01/

[6] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 1–22, 2013.

[7] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[8] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[9] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[10] Y. Du, F. Wei, and H. Zhang, "AnyTool: Self-reflective, hierarchical agents for large-scale API calls," in *Proc. International Conference on Machine Learning (ICML)*, 2024.

[11] X. Liu, Z. Peng, X. Yi, X. Xie, L. Xiang, Y. Liu, and D. Xu, "ToolNet: Connecting large language models with massive tools via tool graph," *arXiv preprint arXiv:2403.00839*, 2024.

[12] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J.-R. Wen, "COLT: Towards completeness-oriented tool retrieval for large language models," in *Proc. 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, 2024, pp. 1930–1940.

[13] E. Lumer, A. Gulati, V. K. Subbiah, P. H. Basavaraju, and J. A. Burke, "ScaleMCP: Dynamic and auto-synchronizing Model Context Protocol tools for LLM agents," *arXiv preprint arXiv:2505.06416*, 2025.

[14] X. Fei, X. Zheng, and H. Feng, "MCP-Zero: Active tool discovery for autonomous LLM agents," *arXiv preprint arXiv:2506.01056*, 2025.

[15] T. Gan and Q. Sun, "RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation," *arXiv preprint arXiv:2505.03275*, 2025.

[16] K.-H. Huang, A. Prabhakar, O. Thorat, D. Agarwal, P. K. Choubey, Y. Mao, S. Savarese, C. Xiong, and C.-S. Wu, "CRMArena-Pro: Holistic assessment of LLM agents across diverse business scenarios and interactions," *arXiv preprint arXiv:2505.18878*, 2025.

[17] Z. Luo, Z. Shen, W. Yang, Z. Zhao, P. Jwalapuram, A. Saha, D. Sahoo, S. Savarese, C. Xiong, and J. Li, "MCP-Universe: Benchmarking large language models with real-world Model Context Protocol servers," *arXiv preprint arXiv:2508.14704*, 2025.

[18] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow, "MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers," *arXiv preprint arXiv:2508.20453*, 2025.

[19] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[20] N. Hardy, "The confused deputy: (or why capabilities might have been invented)," *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988.

[21] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[22] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 14210–14227.

[23] B. Campbell, C. Mortimore, and M. Jones, "Security Assertion Markup Language (SAML) 2.0 Profile for OAuth 2.0 Client Authentication and Authorization Grants," Internet Engineering Task Force (IETF), RFC 7522, May 2015.

[24] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore, "OAuth 2.0 Token Exchange," Internet Engineering Task Force (IETF), RFC 8693, Jan. 2020.

[25] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[26] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," *arXiv preprint arXiv:2505.23643*, 2025.

[27] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," *arXiv preprint arXiv:2504.11703*, 2025.

[28] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li, "Taming various privilege escalation in LLM-based agent systems: A mandatory access control framework," *arXiv preprint arXiv:2601.11893*, 2026.

[29] N. Maloyan and D. Namiot, "Breaking the protocol: Security analysis of the Model Context Protocol specification and prompt injection vulnerabilities in tool-integrated LLM agents," *arXiv preprint arXiv:2601.17549*, 2026.

[30] Y. Song, W. Xiong, D. Zhu, W. Wu, H. Qian, M. Song, H. Huang, C. Li, K. Wang, R. Yao, Y. Tian, and S. Li, "RestGPT: Connecting large language models with real-world RESTful APIs," *arXiv preprint arXiv:2306.06624*, 2023.

[31] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[32] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, "An LLM compiler for parallel function calling," in *Proc. International Conference on Machine Learning (ICML)*, 2024.

[33] E. Y. Chang and L. Geng, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *arXiv preprint arXiv:2503.11951*, 2025.

[34] S. G. Patil, T. Zhang, V. Fang, N. C., R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, "GoEX: Perspectives and designs towards a runtime for autonomous LLM applications," *arXiv preprint arXiv:2404.06921*, 2024.

[35] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.

[36] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," *ACM Transactions on Database Systems (TODS)*, vol. 17, no. 1, pp. 94–162, 1992.

[37] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[38] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," *National Institute of Standards and Technology (NIST)*, Special Publication 800-207, 2020.

[39] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.

[40] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *Proc. International Conference on Learning Representations (ICLR)*, 2025.

[41] P. Hunt, K. Grizzle, M. Ansari, E. Wahlstroem, and C. Mortimore, "System for Cross-domain Identity Management: Protocol," Internet Engineering Task Force (IETF), RFC 7644, Sep. 2015.

[42] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*. Springer, 1972, pp. 85–103.